

Rejected Signatures' Challenges Pose New Challenges: Key Recovery of CRYSTALS-Dilithium via Side-Channel Attacks

Yuan Yuan Zhou^{1*} , Weijia Wang^{2,3}  , Yiteng Sun^{2,3}  and Yu Yu^{4,5}  

¹ Synopsys, Eindhoven, The Netherlands zhou.yuanyuan@gmail.com

² Shandong University, School of Cyber Science and Technology, Qingdao, China
wjwang@sdu.edu.cn, sunyiteng@mail.sdu.edu.cn

³ Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao, China

⁴ Shanghai Jiao Tong University, Shanghai 200240, China yuyu@yuyu.hk

⁵ Shanghai Qi Zhi Institute, 701 Yunjin Road, Shanghai 200232, China

Abstract. Rejection sampling is a crucial security mechanism in lattice-based signature schemes that follow the Fiat-Shamir with aborts paradigm, such as ML-DSA/CRYSTALS-Dilithium. This technique transforms secret-dependent signature samples into ones that are statistically close to a secret-independent distribution (in the random oracle model). While many side-channel attacks have directly targeted sensitive data such as nonces, secret keys, and decomposed commitments, fewer studies have explored the potential leakage associated with rejection sampling. Notably, at HOST 2021, Karabulut et al. showed that leakage from rejected signatures' challenges can undermine, but not entirely break, the security of the Dilithium scheme.

Motivated by the above, we convert the problem of key recovery (from the leakage of rejection sampling) to an integer linear programming problem (ILP), where rejected responses of unique Hamming weights set upper/lower constraints of the product between the challenge and the private key. We formally study the worst-case complexity of the problem as well as empirically confirm the practicality of the rejected signature's challenge attack. For all three security levels of Dilithium-2/3/5, our attack recovers the private key in seconds or minutes with a 100% Success Rate (SR).

Our attack leverages knowledge of the rejected signature's challenge and response, and thus we propose methods to extract this information by exploiting single-trace side-channel leakage from Number Theoretic Transform (NTT) operations and functions associated with the response generation procedure. We demonstrate the practicality of this rejected signature's challenge attack by using real power consumption on an ARM Cortex-M4 microcontroller. To the best of our knowledge, it is the first practical and efficient side-channel key recovery attack on ML-DSA/Dilithium that targets the rejection sampling procedure. Furthermore, we discuss some countermeasures to mitigate this security issue.

Keywords: Dilithium · ML-DSA · Side-channel attacks · Rejection sampling · Integer Linear Programming.

*This work was done when the author worked at SGS Brightsight BV, Delft, The Netherlands.

1 Introduction

1.1 Side-channel attacks on Dilithium/ML-DSA

Side-channel attacks (SCA) [Koc96, KJJ99] exploit physical information leaked during the executions of cryptographic algorithm implementations. Unlike classical cryptanalytic techniques, which focus on the mathematical structure of the algorithms, SCA takes advantage of unintended physical emissions such as power consumption, electromagnetic radiation, or timing information. Since most cryptographic algorithms were not designed with physical information leakage in mind, SCA can be more effective and powerful. For example, it has been shown that SCA can successfully compromise the implementations of almost all well-known cryptographic algorithms.

The National Institute of Standards and Technology (NIST) has recently standardized CRYSTALS-Dilithium (or Dilithium for short) [DKL⁺18] and has named it the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) [NIS24]. Its security has been thoroughly studied from reductionist and cryptanalytic perspectives [DKL⁺18, KLS18, LZ19, DFMS19, BDK⁺20, DFPS23, BBD⁺23, JMW24], particularly in the (classical or quantum) Random Oracle Model (ROM or QROM). As expected, Dilithium is vulnerable to side-channel attacks, given additional information about the secret signing key.

Algorithm 1 Dilithium Sign [DKL⁺18] (Simplified)

Input: $sk = (\rho, K, tr, s_1, s_2, t_0)$, message M , $pk = (\rho, t_1)$

Output: (z, c)

```

1:  $z \leftarrow \perp$ 
2: while  $z = \perp$  do
3:    $y \xleftarrow{\$} \tilde{S}_{\gamma_1}^\ell$ 
4:    $w \leftarrow Ay$ 
5:    $w_1 \leftarrow \text{HighBits}_q(w)$   $\triangleright w = w_1 \cdot 2\gamma_2 + w_0$ 
6:    $c \leftarrow H(\mu \| w_1 \text{Encode}(w_1))$ 
7:    $z \leftarrow y + cs_1$   $\triangleright$  Compute  $cs_1$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{s}_1)$ 
8:   if  $\text{CheckBound}_1(z) == \text{False}$  or  $\text{CheckBound}_2(w - cs_2) == \text{False}$  then
9:      $z \leftarrow \perp$ 
10: return  $\sigma = (z, c)$ 

```

Algorithm 1 provides an overview of the Dilithium signing process, where the subroutines CheckBound_1 and CheckBound_2 verify whether or not z and $w - cs_2$ meet the respective constraints and output the corresponding Boolean values. The ‘keep sampling until the constraints are met’ process, known as rejection sampling, is essential for ensuring the correctness and security of the scheme. Most SCAs on Dilithium focus on the leakage of its signing process. Table 1 provides a summary of existing SCAs on Dilithium, categorized by attack type and the intermediate values they target.

Ravi et al. [RJH⁺18] first published a SCA attack on Dilithium. They specifically targeted the leakage during the point-wise multiplication of cs_1 and recovered the secret s_1 using differential power analysis. They further demonstrated that recovering s_1 enables the forgery of signatures. This approach of exploiting point-wise multiplication has since been extended to other components, such as cs_2 and ct_0 , and has been further optimized in subsequent literature [CKA⁺21, SLKG23, WGL⁺24, LLZ⁺24, QLZ⁺23b, TS24, TMS24].

Another attack strategy directly recovers the secrets s_1 and s_2 . Kim et al. [KLH⁺20] demonstrated that they could recover the secret by constructing a deep learning-based template. However, in many cases, side-channel analysis only provides partial information about the secret due to noise, making direct recovery of the secret challenging. A more practical approach is to recover sensitive intermediate values being more susceptible to side-channel leakage first and then use multiple such intermediates to reconstruct the

Table 1: Summary of the types and targets of side-channel attacks on Dilithium, where NTT refers to the Number Theoretic Transform, and SPA/CPA refers to the Simple/Correlation Power Analysis, respectively.

	Target	Attack Type
This work	c & $\mathbf{z}$	Profiled SPA
[BAE ⁺ 24, HNP25]	$\mathbf{y}$ & cs_1	Profiled SPA
[BVC ⁺ 23]	$\mathbf{w}_0$	Profiled SPA
[LZS ⁺ 21, UMTS22]	$\mathbf{y}$	Profiled SPA
[QLZ ⁺ 23a, QLZ ⁺ 24]	$\mathbf{y}$ & cs_1	Profiled SPA
[RJH ⁺ 18]	cs_1	CPA
[KLH ⁺ 20, CKA ⁺ 21, QLZ ⁺ 23b, SLKG23, WGL ⁺ 24, LLZ ⁺ 24, TS24, TMS24]	NTT(cs_1), NTT(cs_2)	CPA
[WNGD23]	s_1, s_2	Profiled SPA

secret key. For example, Marzougui et al. [UMTS22] proposed a method to distinguish signatures in which one coefficient in $\mathbf{y}$ is 0 based on side-channel leakage. By obtaining multiple such signatures, they were able to recover the secret. Qiao et al. [QLZ⁺23a] demonstrated that one bit in the coefficients of $\mathbf{y}$ is sufficient for key recovery, at the cost of using many more signatures. Berzati et al. [BVC⁺23] extended this strategy by focusing on the leakage of $\mathbf{w}_0$.

However, in the literature, little attention has been paid to the leakage of public data, such as the rejected signature’s challenge c . It is worth noting that in [KAA21] Karabulut et al. targeted the challenge c and recovered the signs of its nonzero coefficients. They concluded that this did not compromise the overall security of Dilithium but resulted in a reduction of τ bits of entropy in c , where τ is the number of nonzero coefficients in c . Moreover, Bronchain et al. [BAE⁺24] explored attacks targeting the rejected response $\mathbf{z}$ and $\mathbf{y}$. Based on simulation experiments, they show that key recovery of s_1 is feasible assuming the knowledge of 32 Hamming weight (HW) classes of all coefficients of $\mathbf{y}$ and the index of the rejected coefficient of $\mathbf{z}$ in an Early-Abort setting. It requires about 10^7 rejected $\mathbf{z}$ values to recover s_1 for Dilithium-2 with a relatively high signal-to-noise ratio (SNR) of 1 and an even higher SNR of 10 for Dilithium-3/5. Recently, Hermelink et al. validated the the method proposed in [BAE⁺24] on physical devices.

In the Dilithium scheme, the challenge c is generally considered public randomness (produced by a random oracle) and thus is considered benign from a leakage perspective. As a result, no specific side-channel countermeasures have been recommended [ABC⁺23], nor have any been implemented in open-source libraries such as the Dilithium reference implementation [KPR⁺], Botan library’s [Bot24] Dilithium implementation, or the ML-DSA implementation in the OQS (Open Quantum Safe) library (a provider of the widely used OpenSSL library) [Lib24]. In addition, it is well known that the rejected response $\mathbf{z}$ can exhibit a bias related to the private key s_1 , as discussed in the specification document [DKL⁺18, NIS24]. It is recommended in [ABC⁺23] to keep $\mathbf{z}$ protected until it passes the bound check to mitigate its potential SCA leakage. However, no published SCA attacks have specifically targeted $\mathbf{z}$. Furthermore, the proposed masking countermeasure [ABC⁺23, CGL⁺24] to protect $\mathbf{z}$ is rather costly, with first-order masking that slows down operations by approximately 40 times according to the state-of-the-art results [CGL⁺24], which is particularly challenging for embedded devices. In contrast, developers are well aware that they must adequately protect other targeted variables $\mathbf{y}, \mathbf{w}_0, s_1, s_2, cs_1, cs_2, \text{NTT}(cs_1)$, and $\text{NTT}(cs_2)$ in the literature, as shown in Table 1 against SCA attacks [DKL⁺18, NIS24, ABC⁺23, RCDB24].

1.2 Our contribution

In this work, we aim to answer the question: *how and to what extent can we recover the Dilithium private key by combining the implementation leakages of rejected signature’s response $\mathbf{z}$ and its corresponding challenge c theoretically and empirically?* Our contributions

in this context are summarized as follows.

First, we formalize the problem of key recovery from the leakages of rejected signature's response and challenge. Specifically, we observe that rejections are caused by out-of-bound coefficients in $\mathbf{z}$. Our attack proceeds in two main steps: (1) recover a number of rejected $\mathbf{z}$ (especially those with unique Hamming weights favored by side-channel attacks) along with their corresponding challenges c , and (2) construct a system of inequalities involving the private key $\mathbf{s}_1$. We then study the worst-case complexity of the key recovery problem given a linear number of these inequalities (signature generations). We also empirically determine the required number of signatures and the bounds for all three NIST security levels 2, 3, and 5 for the proposed rejected signature's challenge attack. We show that the 3×10^6 (resp., 2.3×10^7 and 1.3×10^7) signature generation trials are sufficient for our attack to recover $\mathbf{s}_1$ in seconds or minutes for security level 2 (resp., 3 and 5) with a 100% SR.

Second, since the rejected signature's challenge attack requires knowledge of the rejected signature's challenge c and the corresponding response $\mathbf{z}$, we propose methods to extract these information. Our attack on rejected signature's challenge c focuses on NTT operations that are particularly vulnerable to side-channel attacks due to the intensive use of sensitive values. And, our attack on the corresponding $\mathbf{z}$ focuses on functions associated with the response generation procedure. These enable the extraction of the c and $\mathbf{z}$ with 100% accuracy from single-trace leakage. We experimentally verify the feasibility and validity of our rejected signature's challenge attack using deep learning-based side-channel leakage from a typical real-world target, namely, a reference Dilithium-2 implementation on a 32-bit STM32F microcontroller. The experimental results demonstrate the successful recovery of both the rejected signature's challenge and the response with a 100% accuracy rate.

As a takeaway message, our work highlights that existing countermeasures for side-channel leakages in other operations are inadequate. Multiple proposals exist for masking the bound check on $\mathbf{z}$, but all of these schemes introduce significant overhead. This additional overhead arises from the inevitable conversion between arithmetic and Boolean operations (see, e.g., the construction of SecA2BModp in [ABC⁺23]). For instance, Azouaoui et al. [ABC⁺23] report that the second-order masked bound check for Dilithium-3 requires 4534 kcycles, while the masked NTT only takes 185 kcycles. This disparity highlights the particular challenges of protecting the bound check on embedded devices. In addition, Coron et al. [CGL⁺24] demonstrate that first-order masking slows down operations approximately 40 times. We must also focus on rejection sampling procedures. We discuss some potential countermeasures, such as masking and hiding techniques.

1.3 Related works

Azevedo-Oliveira et al. recently proposed a fault attack against Dilithium [AOVCG25] that builds on the premise that the bound check could be skipped (by, e.g., injecting faults), enabling the recovery of the secret with a sufficient number of samples. Despite the shared similarity with this concurrent work, our attack is non-invasive and exploits only the power leakages corresponding to the challenge and response. Furthermore, we empirically validate the practicality of our attack. Given the ubiquity of power leakage, we believe our attack underscores the critical importance of thoroughly protecting the challenge and response in the implementation of Dilithium.

1.4 Organization of the paper

The rest of this paper is structured as follows. Section 2 introduces background information about rejection sampling and Dilithium. Section 3 formulates the rejected signature's challenge key recovery attack on Dilithium, and provides a formal analysis. Section 4

experimentally verifies the effectiveness and efficiency of this approach. Finally, Section 5 demonstrates the practicality of the rejected signature’s challenge attack by leveraging side-channel leakages from a typical real-world implementation of Dilithium.

2 Preliminaries

2.1 Notations

We adhere to the conventions defined in the NIST FIPS 204 [NIS24] wherever applicable. Let $[a, b]$ denote the intervals of integers $\{a, a + 1, \dots, b\}$ given $a, b \in \mathbb{Z}$. We denote by $\mathcal{N}(0, \sigma^2)$ the normal distribution with mean 0 and standard deviation σ . We denote polynomials by regular lowercase letters (e.g., c), vectors of polynomials by lowercase letters in bold (e.g., $\mathbf{z}$) and matrices of polynomials by uppercase letters in bold (e.g., $\mathbf{A}$). We define the polynomial ring $R_q \stackrel{\text{def}}{=} \mathbb{Z}_q[X]/\langle X^{n+1} \rangle$ with $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$, in which, q is an odd prime number and $n \in \mathbb{N}$, they are set to 8380417 and 256 respectively in the ML-DSA/Dilithium scheme. We represent a polynomial $c = c_0 + c_1X + \dots + c_{n-1}X^{n-1} \in R_q$ as a vector $(c_0, \dots, c_{n-1}) \in \mathbb{Z}_q^n$. Given $r \in \mathbb{Z}_q$, $r \bmod^\pm q$ is defined to be the unique element $r' \in \mathbb{Z}$ such that $-(q-1)/2 \leq r' \leq (q-1)/2$ and $r' \equiv r \bmod q$. For any $c \in R_q$, we define $|c|_i \stackrel{\text{def}}{=} |c_i \bmod^\pm q|$ for all $i \in \{0, 1, \dots, n-1\}$ and $\|c\|_\infty \stackrel{\text{def}}{=} \max_i |c|_i$. Similarly, we define $\|\mathbf{z}\|_\infty \stackrel{\text{def}}{=} \max_{i \in [1, \ell]} \|\mathbf{z}_i\|_\infty$ for any $\mathbf{z} \in R_q^\ell$. A capital alphabet in italics (e.g., S) refers to a finite nonempty set, $s \leftarrow S$ denotes an assignment of an element s from S . Let S_η denote the set $\{s \in R_q \mid \|s\|_\infty \leq \eta\}$ with $\eta \in \mathbb{N}$. Similarly, we denote B_τ as the set $\{b \in R_q \mid \|b\|_\infty = 1\}$ and b has exactly τ nonzero coefficients with $\tau \in \mathbb{N}$, so, $|B_\tau| = 2^\tau \binom{n}{\tau}$. By ‘log’ we denote the base 2 logarithm.

2.2 Rejection sampling in Lyubashevsky’s signature scheme

Schnorr [Sch89] employed the Fiat-Shamir transformation [FS86] to transform a constructed identification scheme into a signature scheme. Lyubashevsky introduced lattice-based signature schemes in [Lyu09, Lyu12] by adapting Schnorr’s approach and incorporating aborts during signature generation. This “Fiat-Shamir with aborts” paradigm aims to abort/discard signatures that could potentially leak secret key information. This rejection sampling [Lyu09] is a cornerstone used to construct many lattice-based signature schemes [Lyu12, BG14], including Dilithium/ML-DSA [DKL⁺18, NIS24]. It produces a distribution that is statistically independent of the secret key in the random oracle model, and thus eliminates the leakage of secret key information.

2.3 Dilithium

Algorithm 2 KeyGen

- 1: $\zeta \leftarrow \{0, 1\}^{256}$
 - 2: $(\rho, \rho', K) \in \{0, 1\}^{256} \times \{0, 1\}^{512} \times \{0, 1\}^{256} \leftarrow H(\zeta, 1024) \triangleright H$ is instantiated as SHAKE-256 throughout
 - 3: $\mathbf{A} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho) \triangleright \mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$
 - 4: $(\mathbf{s}_1, \mathbf{s}_2) \in S_\eta^\ell \times S_\eta^k \leftarrow \text{ExpandS}(\rho')$
 - 5: $\mathbf{t} \leftarrow \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 \triangleright \text{Compute } \mathbf{A}\mathbf{s}_1 \text{ as } \text{NTT}^{-1}(\hat{\mathbf{A}} \cdot \text{NTT}(\mathbf{s}_1))$
 - 6: $(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}_q(\mathbf{t}, 13)$
 - 7: $tr \in \{0, 1\}^{256} \leftarrow H(\rho \parallel \mathbf{t}_1, 256)$
 - 8: **return** $(\text{pk} = (\rho, \mathbf{t}_1), \text{sk} = (\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0))$
-

Dilithium includes three basic procedures: key generation, signature generation, and signature verification. In this subsection, we provide a brief overview of these processes, and refer the readers to official documentation for more details about the scheme. Algorithm 2 illustrates the key generation process of Dilithium. One should pay particular attention to the secret polynomials $\mathbf{s}_1$ and $\mathbf{s}_2$. These polynomials have a small norm such that $\|\mathbf{s}_1\|_\infty \leq \eta$ and $\|\mathbf{s}_2\|_\infty \leq \eta$. All coefficients in these polynomials are considered uniformly and independently drawn (using `ExpandS`).

Algorithm 3 `Sign( $sk, M$ )`

```

1:  $\mathbf{A} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho)$  ▷  $\mathbf{A}$  is generated and stored in NTT representation as  $\hat{\mathbf{A}}$ 
2:  $\mu \in \{0, 1\}^{512} \leftarrow H(tr\|M, 512)$ 
3:  $\kappa \leftarrow 0, (\mathbf{z}, \mathbf{h}) \leftarrow \perp$ 
4:  $\rho' \in \{0, 1\}^{512} \leftarrow H(K\|\mu, 512)$  ▷ or  $\rho' \leftarrow \{0, 1\}^{512}$  for randomized signing
5: while  $(\mathbf{z}, \mathbf{h}) = \perp$  do ▷ Pre-compute  $\hat{\mathbf{s}}_1 \stackrel{\text{def}}{=} \text{NTT}(\mathbf{s}_1)$ ,  $\hat{\mathbf{s}}_2 \stackrel{\text{def}}{=} \text{NTT}(\mathbf{s}_2)$ , and  $\hat{\mathbf{t}}_0 \stackrel{\text{def}}{=} \text{NTT}(\mathbf{t}_0)$ 
6:    $\mathbf{y} \in \tilde{S}_{\gamma_1}^\ell \leftarrow \text{ExpandMask}(\rho', \kappa)$ 
7:    $\mathbf{w} \leftarrow \mathbf{A}\mathbf{y}$  ▷ or  $\mathbf{w} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \cdot \text{NTT}(\mathbf{y}))$ 
8:    $\mathbf{w}_1 \leftarrow \text{HighBits}_q(\mathbf{w}, 2\gamma_2)$ 
9:    $\tilde{c} \in \{0, 1\}^{256} \leftarrow H(\mu\|\mathbf{w}_1, 256)$ 
10:   $c \in B_\tau \leftarrow \text{SampleInBall}(\tilde{c})$  ▷ Store  $c$  in NTT representation as  $\hat{c} \stackrel{\text{def}}{=} \text{NTT}(c)$ 
11:   $\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{s}_1$  ▷ Compute  $c\mathbf{s}_1$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{s}}_1)$ 
12:   $\mathbf{r}_0 \leftarrow \text{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$  ▷ Compute  $c\mathbf{s}_2$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{s}}_2)$ 
13:  if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$  or  $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$  then
14:     $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ 
15:  else
16:     $\mathbf{h} \leftarrow \text{MakeHint}_q(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$  ▷ Compute  $c\mathbf{t}_0$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{t}}_0)$ 
17:    if  $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$  or the number of 1's in  $\mathbf{h}$  is greater than  $\omega$  then
18:       $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ 
19:     $\kappa \leftarrow \kappa + \ell$ 
20: return  $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ 

```

The signing process depicted in Algorithm 3 generates a masking vector of polynomials $\mathbf{y}$ with coefficients bounded by γ_1 . The signer then computes $\mathbf{w} = \mathbf{A}\mathbf{y}$ and extracts $\mathbf{w}_1$, representing the “high-order” bits of the coefficients in this vector. Specifically, each coefficient in $\mathbf{w}$ can be decomposed canonically as $\mathbf{w} = \mathbf{w}_1 \cdot 2\gamma_2 + \mathbf{w}_0$, where $\|\mathbf{w}_0\|_\infty \leq \gamma_2$. Subsequently, a challenge c is generated as the hash of the message and $\mathbf{w}_1$. The resulting c is a polynomial in R_q with exactly τ coefficients of ± 1 's, and the rest being 0. This distribution is designed in such a way that c has a small norm while still coming from a domain with a size greater than 2^{256} . Finally, the signer computes $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ as the potential signature.

Rejection sampling is used to eliminate the dependency of $\mathbf{z}$ on the secret key. The parameter β is the maximum absolute value of any coefficient in $c\mathbf{s}_i$. Given that c has τ coefficients of ± 1 's and the maximum absolute value of any coefficient in $\mathbf{s}_i$ is η , we have $\|c\mathbf{s}_i\|_\infty \leq \beta = \tau\eta$. $\mathbf{y}$ is in the range of $[-\gamma_1 + 1, \gamma_1]$, $c\mathbf{s}_1$ is in the range of $[-\beta, \beta]$, a conservative approach was taken in Dilithium to define the upper/lower bound of $\mathbf{z}$ regarding the rejection sampling, that is, taking the lower bound of $\mathbf{y}$ with the upper bound of $c\mathbf{s}_1$ as the lower bound $-\gamma_1 + 1 + \beta$, taking the upper bound of $\mathbf{y}$ with the lower bound of $c\mathbf{s}_1$ as the upper bound $\gamma_1 - \beta$. If the absolute value of any coefficient of $\mathbf{z}$ exceeds $\gamma_1 - \beta$, the signing procedure is rejected and is restarted. Likewise, if the absolute value of any coefficient of “low-order” bits of $\mathbf{A}\mathbf{z} - c\mathbf{t}$ exceeds $\gamma_2 - \beta$, the procedure is also restarted. The first check is necessary for security, and the second is essential for security and correctness. The signing repeats until both conditions are satisfied.

Dilithium's verification process is depicted in Algorithm 4. It computes $\mathbf{w}'_1$, which represents the “high-order” bits of $\mathbf{A}\mathbf{z} - c\mathbf{t}$. The verifier accepts the signature if all

Algorithm 4 $\text{Verify}(pk, M, \sigma)$

- 1: $\mathbf{A} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho)$ $\triangleright \mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$
 - 2: $\mu \in \{0, 1\}^{512} \leftarrow \text{H}(\text{H}(\rho \| \mathbf{t}_1) \| M, 512)$
 - 3: $c \leftarrow \text{SampleInBall}(\tilde{c})$
 - 4: $\mathbf{w}'_1 \leftarrow \text{UseHint}_q(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$ $\triangleright$ Compute as
 $\text{NTT}^{-1}(\hat{\mathbf{A}} \cdot \text{NTT}(\mathbf{z}) - \text{NTT}(c) \cdot \text{NTT}(\mathbf{t}_1 \cdot 2^d))$
 - 5: **return** $[\|\mathbf{z}\|_\infty < \gamma_1 - \beta]$ and $[\tilde{c} = \text{H}(\mu \| \mathbf{w}'_1)]$ and $[\# \text{ of 1's in } \mathbf{h} \leq \omega]$
-

coefficients of $\mathbf{z}$ fall within the range of $[-\gamma_1 + \beta + 1, \gamma_1 - \beta - 1]$ and if c matches the hash of the message and $\mathbf{w}'_1$. Since $\mathbf{A}\mathbf{z} - c\mathbf{t} = \mathbf{A}\mathbf{y} - c\mathbf{s}_2$, it suffices to show that $\text{HighBits}_q(\mathbf{A}\mathbf{y}, 2\gamma_2) = \text{HighBits}_q(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)$. The reason is that a valid signature ensures $\|\text{LowBits}_q(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)\|_\infty < \gamma_2 - \beta$. Given that the coefficients of $c\mathbf{s}_2$ are upper-bounded by β , adding $c\mathbf{s}_2$ cannot cause any low-order coefficient to exceed γ_2 by introducing carry bits. Thus, the above equations and inequalities hold, and the signature is verified correctly.

Dilithium has three security levels, namely, Dilithium-2/3/5 [BDK⁺20], which correspond to the ML-DSA security levels ML-DSA-44/65/87 [NIS24]. Table 2 summarizes the parameter choices related to this attack for each security level.

 Table 2: Relevant ML-DSA/Dilithium parameter choices [BDK⁺20, NIS24]

Scheme/NIST security level	(k, ℓ)	η	τ	$\beta (= \eta \cdot \tau)$	γ_1	#Reps
ML-DSA-44 (L2)	(4,4)	2	39	78	2^{17}	4.25
ML-DSA-65 (L3)	(6,5)	4	49	196	2^{19}	5.1
ML-DSA-87 (L5)	(8,7)	2	60	120	2^{19}	3.85

2.4 Integer Linear Programming (ILP)

Following Lenstra’s work [Jr.83], we consider a variant of the ILP problem as follows: for integral matrix $\mathbf{A} \in \mathbb{Z}^{m \times n}$ and vector $\mathbf{b} \in \mathbb{Z}^m$, find a vector $\mathbf{x} \in \mathbb{Z}^n$ that satisfies the system of m inequalities $\mathbf{A}\mathbf{x} \leq \mathbf{b}$.

In general, the ILP problem is *NP-complete* [GJ79, vzGS78]. Lenstra [Jr.83] showed that this problem is polynomially solvable with a time complexity of $2^{O(n^3)} \cdot (m \cdot \log V)^{O(1)}$ when the number of variables n is a constant, where V represents the maximum absolute value of the coefficients in $\mathbf{A}$ and $\mathbf{b}$. More recently, in [RR23, Theorem 41] Reis et al. improved this time complexity to $(\log n)^{O(n)} \cdot (m \cdot \log V)^{O(1)}$.

3 Rejected signature’s challenge attack on Dilithium

As discussed in Section 2.2, the rejection sampling of Dilithium guarantees that those $\mathbf{z}$ that leak substantial information about the private key are rejected and not produced as output. In the context of side-channel attack, our objective is to recover the private key $\mathbf{s}_1$ using implementation leakages of the rejected $\mathbf{z}$ and the corresponding c .

As shown in Algorithm 3, $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and the coefficients of the mask polynomial vector $\mathbf{y}$ are uniformly distributed in the interval $[-\gamma_1 + 1, \gamma_1]$, the challenge polynomial c has precisely τ nonzero coefficients (-1 or 1), the secret key polynomial vector has a small norm such that $\|\mathbf{s}_1\|_\infty \leq \eta$ with each coefficient drawn from a discrete uniform distribution. Moreover, we recall a basic property of polynomial multiplication modulo $X^n + 1$. That is, polynomial multiplication $c\mathbf{s}_1$ results in a polynomial $\mathbf{x}$ with each coefficient defined as $\mathbf{x}_i = \sum_{j=0}^i c_{i-j}\mathbf{s}_{1j} - \sum_{j=i+1}^{n-1} c_{n+i-j}\mathbf{s}_{1j}$ with the representation of cyclotomic transformation of c as illustrated in Equation (1). Concretely, it implements the expression $\mathbf{x}_i = \sum_j c'_j \mathbf{s}_{1j}$ where c is rotated in a nega-cyclic manner to get the polynomial

$c' = \text{rot}(c, i)$ defined as $\text{rot}(c, i) = c \cdot X^i \bmod X^n + 1$, i.e., the i -th row of the matrix in Equation (1). Then, denoted by $\mathbf{x}_i$ the $(i + 1)$ -th coefficient of the product $c\mathbf{s}_1$, which is given by the inner product $\mathbf{x}_i = \langle c_i, \mathbf{s}_1 \rangle$.

$$\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_{n-2} \\ \mathbf{x}_{n-1} \end{bmatrix} \stackrel{\text{def}}{=} c\mathbf{s}_1 = \begin{bmatrix} c_0 & -c_{n-1} & -c_{n-2} & \cdots & -c_2 & -c_1 \\ c_1 & c_0 & -c_{n-1} & \cdots & -c_3 & -c_2 \\ c_2 & c_1 & c_0 & \cdots & -c_4 & -c_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ c_{n-2} & c_{n-3} & c_{n-4} & \cdots & c_0 & -c_{n-1} \\ c_{n-1} & c_{n-2} & c_{n-3} & \cdots & c_1 & c_0 \end{bmatrix} \begin{bmatrix} \mathbf{s}_{1_0} \\ \mathbf{s}_{1_1} \\ \mathbf{s}_{1_2} \\ \vdots \\ \mathbf{s}_{1_{n-2}} \\ \mathbf{s}_{1_{n-1}} \end{bmatrix} \quad (1)$$

By the Central Limit Theorem, the distribution of each $\mathbf{x}_i$ approximates $\mathcal{N}(0, \frac{\tau \cdot \eta \cdot (\eta + 1)}{3})$, taking into account that c has exactly τ non-zero coefficients. This equation is straightforward to solve to recover $\mathbf{s}_1$ with known $c\mathbf{s}_1$ and its corresponding challenge c . However, as mentioned above, known SCA countermeasures already suggest protecting secret-involved calculations such as $c\mathbf{s}_1$ and the mask polynomial vector $\mathbf{y}$. We address the problem of recovering $\mathbf{s}_1$ using rejection information alone (without leakage about multiplication $c\mathbf{s}_1$ or mask polynomial vector $\mathbf{y}$).

The probability of $\mathbf{z}$ been rejected is $\Pr[\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta] = 1 - (1 - \frac{\beta}{\gamma_1 - 0.5})^{\ell n} \approx 1 - e^{-\beta \ell n / \gamma_1}$ according to the Dilithium documentation [DKL⁺18]. Given a rejected $\mathbf{z}$, at least one of its coefficients $\mathbf{z}_i$ will be in the interval $[-\gamma_1 + 1 - \beta, -\gamma_1 + \beta] \cup [\gamma_1 - \beta, \gamma_1 + \beta]$, i.e., $4\beta + 1$ possible integral values $\mathbf{z}_i$ in the set defined as $S_z \stackrel{\text{def}}{=} \{-\gamma_1 + 1 - \beta, -\gamma_1 + 2 - \beta, \dots, -\gamma_1 - 1 + \beta, -\gamma_1 + \beta\} \cup \{\gamma_1 - \beta, \gamma_1 - \beta + 1, \dots, \gamma_1 - 1 + \beta, \gamma_1 + \beta\}$. These $4\beta + 1$ possible $\mathbf{z}_i$ values can confine the corresponding $\mathbf{y}_i$ to a much smaller interval than the original $[-\gamma_1 + 1, \gamma_1]$ and thus further limit the range of $\mathbf{x}_i$. That is because, given a rejected coefficient $\mathbf{z}_i$ in S_z , the corresponding $\mathbf{y}_i$ is expected to be bounded by $[\mathbf{z}_i - \beta, \mathbf{z}_i + \beta]$, but this interval is further narrowed down due to its original bound of $[-\gamma_1 + 1, \gamma_1]$. Due to this further confining of $\mathbf{y}_i$, the upper (or lower) bound of the corresponding $\mathbf{x}_i$ is also further confined from β (or $-\beta$) to a lower constraint accordingly. For instance,

- if $\mathbf{z}_i = -\gamma_1$, then $-\gamma_1 + 1 \leq \mathbf{y}_i \leq -\gamma_1 + \beta$ (narrowed down from the expected interval $[-\gamma_1 - \beta, -\gamma_1 + \beta]$) and $-\beta \leq \mathbf{x}_i \leq -1$ (its upper bound confined to -1 from β);
- if $\mathbf{z}_i = -\gamma_1 - 1$, then $-\gamma_1 + 1 \leq \mathbf{y}_i \leq -\gamma_1 - 1 + \beta$ (narrowed down from the expected interval $[-\gamma_1 - 1 - \beta, -\gamma_1 - 1 + \beta]$) and $-\beta \leq \mathbf{x}_i \leq -2$ (its upper bound confined to -2 from β);
- if $\mathbf{z}_i = \gamma_1 - 1$, then $\gamma_1 - 1 - \beta \leq \mathbf{y}_i \leq \gamma_1$ (narrowed down from the expected interval $[\gamma_1 - 1 - \beta, \gamma_1 - 1 + \beta]$) and $-1 \leq \mathbf{x}_i \leq \beta$ (its lower bound confined to -1 from $-\beta$);
- if $\mathbf{z}_i = \gamma_1$, then $\gamma_1 - \beta \leq \mathbf{y}_i \leq \gamma_1$ (narrowed down from the expected interval $[\gamma_1 - \beta, \gamma_1 + \beta]$) and $0 \leq \mathbf{x}_i \leq \beta$ (its lower bound confined to 0 from $-\beta$);
- if $\mathbf{z}_i = -\gamma_1 + 1$, then $-\gamma_1 + 1 \leq \mathbf{y}_i \leq -\gamma_1 - 1 + \beta$ (narrowed down from the expected interval $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$) and $-\beta \leq \mathbf{x}_i \leq 0$ (its upper bound confined to 0 from β);
- if $\mathbf{z}_i = \gamma_1 - 2$, then $\gamma_1 - 1 - \beta \leq \mathbf{y}_i \leq \gamma_1$ (narrowed down from the expected interval $[\gamma_1 - 2 - \beta, \gamma_1 - 2 + \beta]$) and $-2 \leq \mathbf{x}_i \leq \beta$ (its lower bound confined to -2 from $-\beta$).

With the aforementioned upper/lower constraints of $\mathbf{x}_i$, we can construct an ILP instance of $\mathbf{x}_i$ with (ℓn) variables (the polynomial degree parameter n is 256 and the ℓ parameter of $\mathbf{s}_1$ set to 4/5/7 for security level 2/3/5 in ML-DSA/Dilithium respectively).

That is, for a confined $\mathbf{x}_i$ in the interval $[\text{lb}, \text{ub}]$, its upper bound ub is used to construct one inequality of an ILP instance as $\mathbf{x}_i \leq \text{ub}$, its lower bound lb is used to construct another inequality of this ILP instance as $-\mathbf{x}_i \leq \text{lb}$. In this way, each confined $\mathbf{x}_i$ contributes two inequalities to this constructed ILP instance. By solving this ILP problem $\mathbf{Ax} \leq \mathbf{b}$ with $\mathbf{x} = \mathbf{s}_1$ in this constructed ILP instance, we can recover $\mathbf{s}_1$ given linearly many pairs of rejected $(\mathbf{z}_i, c)$.

Our main result of the rejected signature's challenge attack on $\mathbf{s}_1$ is as follows:

Definition 1 (Rejected coefficient of a rejected signature's response). For a rejected signature's response $\mathbf{z}$, its rejected coefficient is a coefficient in $\mathbf{z}$ such that $|\mathbf{z}_i| \geq \gamma_1 - \beta$. Moreover, i is referred to as the position of the rejected coefficient.

Theorem 1 (ILP-based Dilithium key recovery). *For the Dilithium signing (Algorithm 3), given $m > \ell n$ rejected signatures' responses $\mathbf{z}$ and the corresponding values and positions of their rejected coefficients $\mathbf{z}_i$ and the corresponding challenges c , an adversary can recover $\mathbf{s}_1$ with probability of at least $1 - \ell n \cdot e^{-\frac{\tau m}{\ell n}}$ and a time complexity of $(\log \ell n)^{O(\ell n)} \cdot (m \cdot \log \beta)^{O(1)}$.*

Proof. Each rejected coefficient $\mathbf{z}_i$ will assume one of $4\beta + 1$ values in the set S_z . For each rejected coefficient $\mathbf{z}_i$, one can construct two inequalities about the unknown $\mathbf{s}_1$. For example, if the rejected $\mathbf{z}_i = -\gamma_1$, we get two inequalities $-\beta \leq \langle c_i, \mathbf{s}_1 \rangle \leq -1$ according to 3, which can be further written as:

$$\begin{bmatrix} -\text{rot}(c, i) \\ \text{rot}(c, i) \end{bmatrix} \mathbf{s}_1 \leq \begin{bmatrix} \beta \\ -1 \end{bmatrix}$$

Taking into account the m rejected coefficients, we can construct an ILP instance $\mathbf{Cs}_1 \leq \mathbf{b}$ for a $2m \times \ell n$ matrix $\mathbf{C}$ and a $2m \times 1$ vector $\mathbf{b}$, where the number of variables in $\mathbf{s}_1$ is ℓn , and the maximum absolute value of the coefficients in $\mathbf{C}$ and $\mathbf{b}$ is β . According to [RR23, Theorem 41], one can solve this ILP problem in time $(\log \ell n)^{O(\ell n)} \cdot (m \cdot \log \beta)^{O(1)}$ to recover the secret key $\mathbf{s}_1$.

For each rejected $\mathbf{z}_i$, only τ non-zero coefficients in the corresponding c are multiplied with the corresponding $\mathbf{s}_1$ as shown in Equation (1). Thus, each rejected $\mathbf{z}_i$ can only add constraints to the constructed ILP instance about τ coefficients of $\mathbf{s}_1$. To fully recover $\mathbf{s}_1$, all the ℓn coefficients of $\mathbf{s}_1$ must be covered by the rejected coefficients $\mathbf{z}_i$. We denote $E_j (j \in [1, \ell n])$ as the event that those m rejected coefficients $\mathbf{z}_i$ have not bounded the j -th coefficient of $\mathbf{s}_1$. The probability that those m rejected coefficients $\mathbf{z}_i$ have not bounded at least one coefficient of $\mathbf{s}_1$ is $\Pr\left[\bigcup_{j=1}^{\ell n} E_j\right] \leq \sum_{j=1}^{\ell n} \Pr[E_j] = \ell n \cdot (1 - p)^m \approx \ell n \cdot e^{-pm}$, in which, $p = \frac{\tau}{\ell n}$ corresponds to the probability of any coefficient of $\mathbf{s}_1$ bounded by any rejected $\mathbf{z}_i$. Putting all together, the probability that all coefficients of $\mathbf{s}_1$ bounded by that m rejected $\mathbf{z}_i$ is $1 - \Pr\left[\bigcup_{j=1}^{\ell n} E_j\right] \geq 1 - \ell n \cdot e^{-\frac{\tau m}{\ell n}}$. This completes the proof. $\square$

In practice, the coefficient $\mathbf{z}_i$ is typically a 32-bit signed integer using two's complement representation. It is very difficult to recover each and every $4\beta + 1$ values in S_z with 100% SR. Targeting HW values of the rejected coefficients $\mathbf{z}_i$ can significantly reduce the number of classes to identify. There are four particular values $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, and γ_1 in S_z that correspond to unique HW values (detailed in Appendix A), which are easier to extract via SCA leakages. We thus define the following rejected signature's challenge that we will exploit towards a more useful and practical attack.

Definition 2 (Rejected signature's challenge of interest - RCoI). For the Dilithium signing (Algorithm 3), a challenge c is a rejected signature's challenge of interest, if c corresponds to a rejected signature's response $\mathbf{z}$ and its rejected coefficient $\mathbf{z}_i$ equals $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$ or γ_1 .

Lemma 1. *For each generation of the challenge c in Dilithium signing (Algorithm 3), the probability that c is a rejected signature's challenge of interest is $\frac{1 - (1 - \frac{\beta}{\gamma_1 - 0.5})^{\ell n}}{\beta + 0.25}$.*

Proof. Conditioned on the event $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, which occurs with probability

$$\Pr \left[\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta \right] = 1 - \left(1 - \frac{\beta}{\gamma_1 - 0.5} \right)^{\ell n},$$

at least one coefficient of $\mathbf{z}$ is uniform over S_z . That is because, when vector $\mathbf{z}$ is uniformly drawn, each coefficient is uniform over $[-\gamma_1 + \beta + 1, \gamma_1 - \beta - 1]$. By definition, $\mathbf{z}$ is rejected if and only if at least one coefficient of $\mathbf{z}$ lies in the set of $4\beta + 1$ values defined by the set S_z . It thus hits the four values (as in Definition 2) with probability $\frac{4}{4\beta + 1} = \frac{1}{\beta + 0.25}$, which completes the proof. $\square$

Taking into account Lemma 1, we can deduce a special case of Theorem 1 as below based on rejected signature's challenge of interest,

Theorem 2 (Key recovery with RCoI). *For M executions of the Dilithium signing (Algorithm 3) with $M > \ell n u$ and $u = \frac{2\beta + 0.5}{1 - (1 - \frac{\beta}{\gamma_1 - 0.5})^{\ell n}}$, an adversary who obtains the rejected signatures' challenges of interest and the corresponding values and positions of their rejected coefficients can recover $\mathbf{s}_1$ with a probability of at least $(1 - \ell n e^{-\frac{\tau M}{\ell n u}} - e^{-\frac{M}{4u}})$ and a time complexity of $(\log \ell n)^{O(\ell n)} \cdot (\frac{M \log \beta}{u})^{O(1)}$.*

Proof. According to Lemma 1, the expected number of rejected signatures' challenges of interest among M signing executions is $M \cdot \frac{1 - (1 - \frac{\beta}{\gamma_1 - 0.5})^{\ell n}}{\beta + 0.25} = \frac{2M}{u}$. We have by a Chernoff bound that the number of rejected signatures' challenges of interest is at least, $\frac{M}{u}$ except with a probability of $e^{-\frac{M}{4u}}$. Then, it is natural to replace $m = \frac{M}{u}$ in Theorem 1, which leads to recovering $\mathbf{s}_1$ with probability of at least $(1 - \ell n e^{-\frac{\tau M}{\ell n u}} - e^{-\frac{M}{4u}})$ and a time complexity of $(\log \ell n)^{O(\ell n)} \cdot (\frac{M \log \beta}{u})^{O(1)}$. $\square$

4 Experimental results

The time complexity given in Theorem 2 is not of much practical significance because it corresponds to the worst-case scenario and contains a huge exponential factor. To assess the effectiveness of the proposed rejected signature's challenge attack, we first evaluate it to recover the entire private key $\mathbf{s}_1$ considering Dilithium-2/3/5 parameter sets. In our experiments, instead of implementing the algorithm proposed in [RR23], we choose the built-in **branch and bound** [LMSK63] exact algorithm from GLPK MILP solver of SAGE to solve the above ILP. This exact algorithm turns out to be very efficient in practice, especially with effective bounding techniques one can prune away large sections of the tree, dramatically reducing the actual computation time. Our experimental results are in line with this intuition, i.e., the constructed ILP instances are solved in seconds or minutes.

Since the coefficients of $\mathbf{s}_1$ follow the discrete uniform distribution with a mean of 0, we set the objective function of the constructed ILP instance to minimize the sum of all $\mathbf{s}_1$ coefficients (i.e., the variables of the constructed ILP instance). This is equivalent to minimizing the mean value of all coefficients of $\mathbf{s}_1$, which is expected to be close to 0. We emulate the rejected signature's challenge attack by sampling random pairs of Dilithium keys, generating corresponding signatures, and recording only those rejected $\mathbf{z}$ with $\mathbf{z}_i \in \{-\gamma_1, -\gamma_1 - 1, \gamma_1 - 1, \gamma_1\}$ and the corresponding c , where 'i' is the index of the coefficient of $\mathbf{z}$ that fails the bound check. Our experiments consider the bound check of $\mathbf{z}$

from the Early-Abort implementation, which is regarded as the worst-case scenario from the attacker perspective. This is because, in the Early-Abort implementation, only one $\mathbf{z}_i$ can be exploited for each rejected $\mathbf{z}$ (before reaching an early abort). In contrast, in other (for example, constant-timing) implementations, multiple interesting $\mathbf{z}_i$ values may be found in a single rejected $\mathbf{z}$. We summarize the experimental rejected signature’s challenge attack results in Tables 3 and 4. The experiments are repeated 100 times for each setting on randomly generated key pairs, so the success rate “SR” column of full key recovery of $\mathbf{s}_1$ corresponds to the average of 100 experiments. We implemented the experiments in SAGE 10.3 (Ubuntu 22.04.4) with an Intel® Core™ i5-7500 CPU 3.40GHz. All experiments used the default GLPK MILP solver of SAGE with a timeout of 1 hour. Each successful full key recovery of $\mathbf{s}_1$ takes a matter of seconds or minutes. We ran the Dilithium reference implementation (KeyGen and Sign) code in the PC. For each experiment (100 experiments per security level of 2, 3 or 5), a random key pair was generated by the Dilithium KeyGen function, and then we repeatedly invoked the Dilithium Sign function (3M/23M/13M times for security level of 2, 3 or 5, respectively) and recorded the rejected signatures and the corresponding challenge c only if the bound check of $\mathbf{z}$ failed and one of the coefficients of $\mathbf{z}$ corresponds to one of those 4 Hamming Weights. Until 3M/23M/13M times of signing operations, we constructed an ILP instance using the recorded rejected signatures and the corresponding challenges with the identified index ‘i’, and then ran the ILP solver to recover the secret key $\mathbf{s}_1$. We also did the experiments using 1M/2.5M/5M signatures for Dilithium-2, 20M/25M for Dilithium-3, 10M/15M for Dilithium-5. Using 20M signatures for Dilithium-3 we obtained an SR of 83% (17 out of 100 experiments failed to recover the full $\mathbf{s}_1$ with timeout error), that is, with 7×10^6 (20M) more signatures than for Dilithium-5 (13M).

The second column of Table 3 contains four numbers. The first number indicates the number of signatures generated for each random key pair. The second number corresponds to the expected number of rejected signatures. The third and the fourth numbers correspond to the number of rejected signatures used in the related works [AOVCG25] (recovering $\mathbf{s}_2$ via fault injection) and [BAE⁺24] (with $\mathbf{y}$ leakage required), respectively. The work [BAE⁺24] presents results under various SNR conditions; however, it is difficult to identify a directly comparable SNR from [BAE⁺24] due to differences in attack scenarios. Specifically, our results in this section assume that the unique Hamming weights of the rejected coefficients in $\mathbf{z}$ ’s can be recovered with 100% accuracy, even though our attack model already accounts for realistic conditions in which the exact coefficient values may not be recoverable. For a conservative comparison, we choose the case with SNR=100 from their results, which corresponds to an almost noise-free setting. It should be noted that, the larger (or smaller) signature counts shown in Table 4 should not be interpreted as indicating the superiority or inferiority of a method without considering the underlying assumptions and attack models. Specifically, our method only requires leakage from the rejected signature’s challenge c and response $\mathbf{z}$, whereas the attack in [BAE⁺24] depends on the leakage of the secret vector $\mathbf{y}$. Besides, the work [AOVCG25] builds on the premise that the bound check could be perfectly skipped (by, e.g., injecting faults), whereas our method considers the leakage of unique Hamming weights of $\mathbf{z}$ ’s rejected coefficients favored by side-channel attack.

Interestingly, the results suggest that the complexity of this attack does not directly depend on the security levels. On the one hand, higher security levels require more signatures. The SR reaches 100% for Dilithium-2/3/5. On the other hand, Dilithium-3 requires 10^7 more signatures than Dilithium-5. The fact that the expected number of rejected four particular $\mathbf{z}_i$ values depends on the parameters ℓ and β explains this. That is, the probability of at least one (considering the Early-Abort setting in the bound check of the reference implementation) $\mathbf{z}_i$ coefficient rejected is $\Pr[\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta] = 1 - (1 - \frac{\beta}{\gamma_1 - 0.5})^{\ell n}$, approximately, 45.64%, 38.04%, 33.65% for Dilithium-2/3/5, respectively. With higher security levels, the expected number of rejected $\mathbf{z}$ is getting smaller, given the same amount

of signature generation trials. However, the probability of the rejected $\mathbf{z}_i$ coefficient being one of those four values is about $\frac{1}{4\beta+1}$, i.e., $\frac{1}{313}, \frac{1}{785}, \frac{1}{481}$ for Dilithium-2/3/5, respectively. This explains the fact that Dilithium-3 needs more signature generations to get sufficient rejected $\mathbf{z}_i$ compared to Dilithium-5.

Table 3: Our attack results using 4 rejected values of $\mathbf{z}$'s coefficients

	#Sign (#Rejected $\mathbf{z}$)	SR	ILP constraint bounds	Running time (s)
ML-DSA-44 (L2)	3×10^6 (1.37×10^6)	100%	$[-52, -1]; [-52, -2]; [-1, 49]; [0, 49]$	104
ML-DSA-65 (L3)	2.3×10^7 (8.75×10^6)	100%	$[-112, -1]; [-112, -2]; [-1, 111]; [0, 111]$	201
ML-DSA-87 (L5)	1.3×10^7 (4.38×10^6)	100%	$[-63, -1]; [-63, -2]; [-1, 63]; [0, 63]$	75

Table 4: A comparison with [AOVCG25] and [BAE⁺24]

	#Sign (#Rejected $\mathbf{z}$) vs. [AOVCG25] [BAE ⁺ 24]	SR
ML-DSA-44 (L2)	3×10^6 (1.37×10^6) 0.72×10^6 0.7×10^6	100%
ML-DSA-65 (L3)	2.3×10^7 (8.75×10^6) 2.40×10^6 2.1×10^6	100%
ML-DSA-87 (L5)	1.3×10^7 (4.38×10^6) 2.44×10^6 0.7×10^6	100%

The fourth column of Table 3 consists of four pairs of constraints, represented by interval $[lb, ub]$, which correspond to the four respective rejected $\mathbf{z}_i$ values, namely $\mathbf{z}_i = -\gamma_1$, $\mathbf{z}_i = -\gamma_1 - 1$, $\mathbf{z}_i = \gamma_1 - 1$, or $\mathbf{z}_i = \gamma_1$. As discussed, we could have just put the standard constraints $[-\beta, -1]$, $[-\beta, -2]$, $[-1, \beta]$ and $[0, \beta]$ to represent the respective inequalities for the four cases $-\beta \leq \mathbf{x}_i \leq -1$, $-\beta \leq \mathbf{x}_i \leq -2$, $-1 \leq \mathbf{x}_i \leq \beta$, and $0 \leq \mathbf{x}_i \leq \beta$, where β is 78, 196 or 120 for Dilithium-2/3/5 respectively. For further optimization, we observe that smaller absolute values can be used in place of above upper/lower bound $\pm\beta$ without affecting the accuracy. We statistically analyze the distribution of $\mathbf{x}_i$ of a rejected $\mathbf{z}_i$. As depicted in the histogram of Figure 1a (resp., Figure 1b and Figure 1c), we execute 10^9 (resp., 2.5×10^9 and 2×10^9) times of Dilithium-2 (resp., Dilithium-3 and Dilithium-5) signing process, and record 458508787 (resp., 952794746 and 675233816) rejected $\mathbf{z}_i$ and the corresponding values of $\mathbf{x}_i$. As can be observed from the experimental results, the lower bound of $-\beta = -78$ (resp., $-\beta = -196$ and $-\beta = -120$) can be reduced to -52 (resp., -112 and -63) for Dilithium-2 (resp., Dilithium-3 and Dilithium-5), which already bounds all values in our experiments. The case for upper bound β is likewise.

We mention that the above empirical optimization has theoretical support. We can observe that $\mathbf{x}_i$ approximates the $\mathcal{N}(0, \sigma^2 = \frac{\tau \cdot \eta \cdot (\eta + 1)}{3})$ normal distribution as outlined in black in Figures 1a, 1b and 1c. The probability that a value drawn from this distribution falls in the interval $[-5.3\sigma, 5.3\sigma]$, is at least 99.99999% (see e.g., [FSK03]) by substituting concrete values into the parameters. The last column of Table 3 gives the average running time of full key recovery of $\mathbf{s}_1$, which is less than 4 minutes for all three security levels. Note that, Theorem 2 gives the worst-case complexity of an ILP solver (with high time complexity to find the optimal integral solution), while in Table 3 we used an empirical ILP solver based on **branch and bound** exact algorithm, which terminates earlier as soon as a (not necessarily optimal) integral solution is found. An analogous but not identical example is that the SAT problem is NP-hard yet an empirically efficient solver can still demonstrate high performance for most real-world SAT instances, often due to its ability to exploit hidden structural patterns within the problem, even if theoretical analysis might not fully explain its effectiveness. As can be observed, the average running time on Dilithium-5 is faster than for Dilithium-2. Due to the used **branch and bound** algorithm solver, the average running time of 100 experiments for each security level was heavily affected by a few harder ILP instances where the found inequalities were less than the other instances. As the number of inequalities in a constructed ILP instance is affected by the used total number of signatures, the total number (3M for Dilithium-2 and 13M for Dilithium-5) in our experiments is based on a coarse grid search for a 100% SR. For Dilithium-2, we tried

1M/2.5M/3M/5M, but only 10M/13M/15M for Dilithium-5, most likely the 3M is closer to the required minimum number of signatures to reach a SR of 100% for Dilithium-2 than the 13M for Dilithium-5.

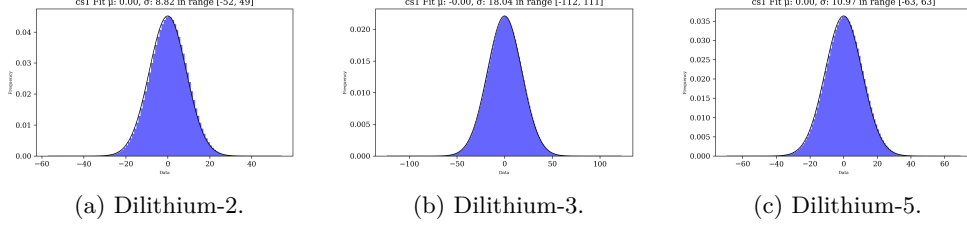


Figure 1: Distributions of $\mathbf{x}_i$ for Dilithium at different security levels.

In all Dilithium-2/3/5 cases, we verified the effectiveness of the proposed rejected signature’s challenge attack using those four particular values of $\mathbf{z}$ ’s rejected coefficient with the corresponding challenge c to disclose the entire private key $\mathbf{s}_1$. Next, we will demonstrate utilizing this rejected signature’s challenge attack using the obtained side-channel leakages of $\mathbf{z}$ ’s coefficients and its corresponding c via profiled attacks in a realistic context.

5 A use case of proposed attack from real SCA leakage

In this section, we validate the practicality and effectiveness of our attack. Specifically, we propose a deep learning-based method to recover the rejected coefficient of $\mathbf{z}$ and the corresponding challenge c from side-channel leakage. By leveraging the proposed rejected signature’s challenge attack, we achieve a successful side-channel key-recovery attack on real implementations.

5.1 Deep learning profiled attack

Since the introduction of template attacks [CRR02], the SCA community has regarded profiled attacks as the most powerful type of side-channel attacks. Profiled attacks consist of two phases, i.e., profiling and attacking phases. In the profiling phase, an attacker uses a profiling device, where he controls or at least knows the key, to model the leakage characteristics of key-dependent sensitive data (e.g., the coefficients of the rejected signature’s challenge c in this work). In the attacking phase, the attacker recovers sensitive information from the target device using side-channel leakage and the model built during the profiling phase.

We employ Deep Learning (DL) profiled attacks to experimentally evaluate the effectiveness of our proposed rejected signature’s challenge attack. Specifically, we use a Multi-Layer Perceptron (MLP) neural network model for these DL profiled attacks. Despite its simple and shallow structure, the MLP model demonstrated strong performance. Unlike the classical profiled attacks (e.g., template attacks), DL profiled attacks do not assume any specific leakage characteristics. They exploit features (sample points from side-channel traces) via neural networks to classify labels (sensitive data in the SCA context). The training process of neural networks in the profiling phase aims to construct a classifier function $F(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^{|S|}$, mapping the input trace $\mathbf{l} \in \mathbb{R}^d$ to an output vector $\mathbf{p} \in \mathbb{R}^{|S|}$, representing classification scores. During training, the backpropagation method [Kel60, GBC16] is applied to each training batch, updating the trainable parameters of the neural network model to minimize the loss function, which quantifies classification errors. In the attacking phase, the trained model (i.e., $F(\cdot)$ with all the final updated

parameters) is used to classify each attack trace and produce a probability vector $\mathbf{p}[s_g]$. We detail the parameter settings for our neural networks in Section 5.2.

5.2 Attack setup

Our target is the reference Dilithium implementation¹ on a 32-bit STM32F microcontroller running at 8 MHz. During the execution of the signing process in Dilithium (before obtaining the signature σ), the values of c and $\mathbf{z}$, whether $\mathbf{z}$ is rejected in the while loop, are unknown to the attacker. We need to recover the corresponding intermediate values through SCA. The following are the leakage points for the corresponding intermediate values.

To implement the attack, we acquired power consumption traces using a Picoscope 5244D oscilloscope and set the sampling rate to 62.5 MHz. The SNR of our setup is 0.34, measured based on loading a variable with value less than the modulus $q = 8\,380\,417$. Specifically, each traces we collected corresponds to a random value of the loaded variable. The SNR was then computed as the ratio between the variance of the group means and the average within-group variance. We need to distinguish between the various types of leakage discussed above, concretely differentiating the values of the coefficients of c , distinguishing some specific values of the coefficients of $\mathbf{z}$, and determining whether the coefficients of $\mathbf{z}$ will be rejected. This task involves a one-dimensional data classification problem. Given that each type of leakage corresponds to a limited number of categories, we found that the representational capacity of an MLP is sufficient. Consequently, we designed an MLP model to perform this classification. Table 5 summarizes the details of the used MLP model and the corresponding hyper-parameters.

Table 5: MLP Model Architecture

Layer Type	Output Size	Details
Input Layer	(input size,)	-
Dense Layer	128	Activation: ReLU, Regularization: L2 (0.01)
Dropout	128	Dropout Rate: 0.2
Output Layer	classes num	Activation: Softmax, Optimizer: Adam

It is a simple MLP structure, where the Dense layer (fully connected layer) is the core component. Each neuron connects to all neurons in the previous layer. We use the ReLU activation function to introduce non-linearity and accelerate the training process. We add L2 regularization to constrain the weight values to reduce the risk of overfitting, along with Dropout, which randomly “disables” a portion of the neurons during training to enhance the model’s generalization ability.

In terms of data preprocessing, all profiling and attacking traces are normalized using the *StandardScalar* function from the Scikit-learn [PVG⁺11] library by removing the mean and scaling to unit variance. For each data point x , the standardized data is $x_{\text{scaled}} = \frac{x - \mu}{\sigma}$, where μ is the mean of the sample data, and σ is the standard deviation of the sample data.

There is an implicit assumption about the knowledge of the Points of Interest (POIs), i.e., the attackers/evaluators can determine the rough timing intervals of each coefficient processing of the rejected signature’s challenge c and the corresponding $\mathbf{z}_i$ in the side-channel traces. It is feasible for most of security products in a grey-box testing context shown in [KAA21, QLZ⁺23a, QLZ⁺24]. In practice, we determine the positions of PoIs by computing the correlation between power traces and the target variables (e.g., the Hamming weight of c ’s coefficients).

¹The specific implementation is detailed in the Dilithium repository on GitHub: <https://github.com/pq-crystals/dilithium>

5.3 Recovering c

c is transformed into its NTT form by employing the Cooley-Tukey (CT) Algorithm. Figure 2 illustrates the NTT code provided by the Dilithium’s reference implementation. The leakage generated by the butterfly operation, highlighted in red, can help us recover c . For the polynomial c with n coefficients $c_0, \dots, c_{n-1}$, this process consists of $\log n$ layers, with $n/2$ CT Butterfly Units (BFUs) repeated for all coefficients in each layer.

```

1  void ntt(int32_t a[N]) {
2      unsigned int len, start, j, k=0; int32_t zeta, t;
3      for(len = 128; len > 0; len >>= 1) {
4          for(start = 0; start < N; start = j + len) {
5              zeta = zetas[++k];
6              for(j = start; j < start + len; ++j) {
7                  /*Attack Position*/
8                  t = montgomery_reduce((int64_t)zeta * a[j + len]);
9                  a[j + len] = a[j] - t;
10                 a[j] = a[j] + t;
11             }
12         }
13     }
14 }
    
```

Figure 2: C Implementation of NTT.

Each CT Bfu operates on two coefficients, referred to as *upper and lower coefficients*, and produces two outputs involving a twiddle factor. Specifically, the product between the upper coefficient and the twiddle factor is added to (or subtracted from) the lower coefficient to generate the results.

Figure 3 provides a brief summary of the NTT process with 16 coefficients. Our attack is to recover the entire challenge c by exploiting the leakage from NTT. In the rest of this subsection, we denote each coefficient in the input and output of the NTT as c_i and $\hat{c}_i$, respectively, with $i \in [0, n-1]$, and we call them as *input or output coefficients*. Additionally, we denote each coefficient in the j^{th} layer as $c_{i,j}$ with $i \in [0, n-1]$ and we call them as *inner coefficients*, or, *coefficients for the sake of brevity*. Moreover, for an upper coefficient $c_{i,j}$ in the j^{th} layer, we denote the corresponding product of $c_{i,j}$ and the associated twiddle factor as $c'_{i,j} = c_{i,j} \cdot \zeta$.

5.3.1 Exploiting leakage of the first layer

First, we can recover the upper input coefficient of each Bfu in the first layer with perfect accuracy. Because the possible values of the upper input coefficient, namely -1 , 0 , and 1 , produce significantly distinct leakage patterns. These patterns are related to the leakages of the upper input coefficient, denoted as c_i and $c_i \cdot \zeta$, where $\frac{n}{2} \leq i \leq n-1$ and $\zeta = 25847$. We focus on the Hamming weight leakage and present the Hamming weights of c_i and $c_i \cdot \zeta$ below.

- When $c_i = -1$, the Hamming weights are 32 and 14, respectively.
- When $c_i = 0$, the Hamming weights are 0 and 0, respectively.
- When $c_i = 1$, the Hamming weights are 1 and 19, respectively.

Additionally, we can distinguish each lower input coefficient value of -1 from 0 and 1 in the first layer with perfect accuracy. Because the Hamming weight of -1 differs significantly from those of 0 and 1 . However, distinguishing between 0 and 1 for lower coefficients is more challenging in these cases—particularly compared to upper coefficients—since the

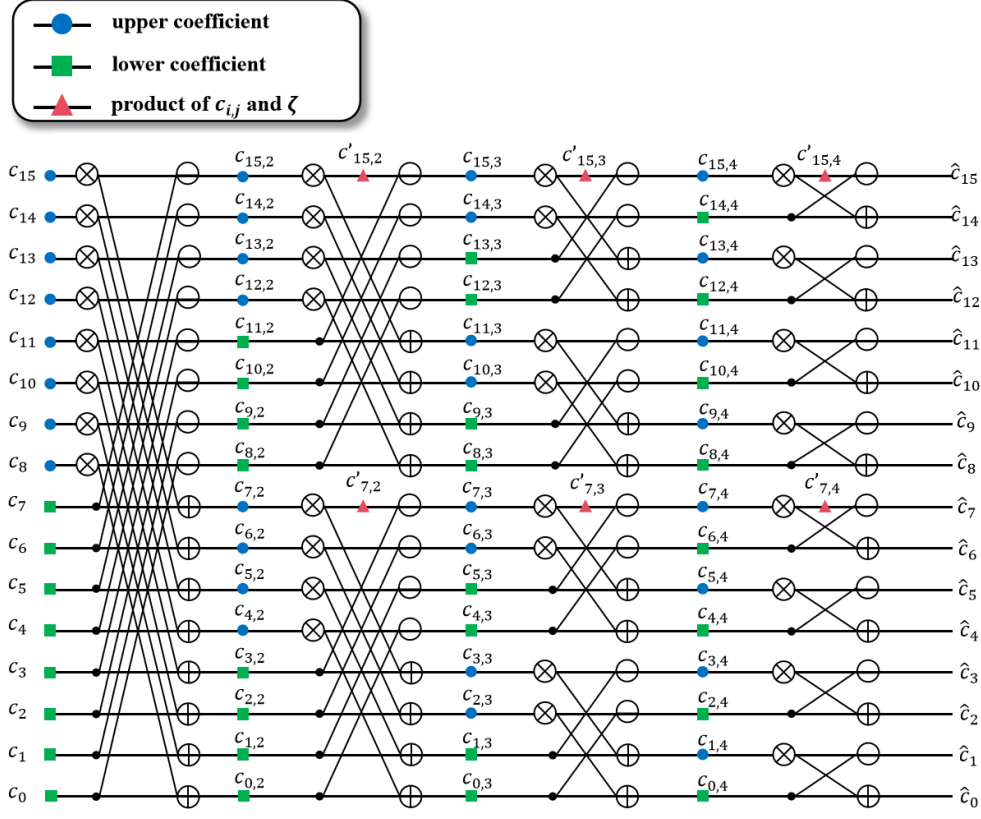


Figure 3: 16×16 BFU construction.

NTT operation at this stage involves only a multiplication between an upper coefficient and the ζ .

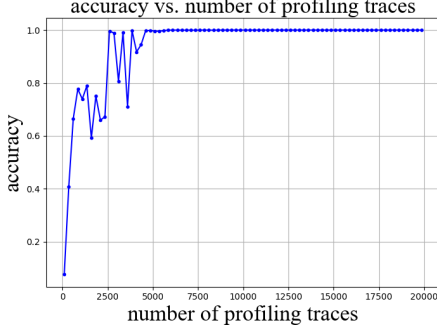
Figure 4 presents the attack results for both the upper and lower input coefficients in the first layer of the NTT. Figures 4a and 4c illustrate the model's (epochs=4, batch size=512) prediction accuracy with the number of profiling traces for the upper and lower input coefficients, respectively. For the upper input coefficients, the accuracy is measured by distinguishing between -1 , 0 , and 1 . At the same time, for the lower input coefficients, the task is to distinguish -1 from $\{0, 1\}$.

We also provide the confusion matrix to describe the accuracy. Each element in the confusion matrix represents the model's prediction and the actual class matching for a specific category. We can observe from Figures 4b and 4d that the prediction accuracy has reached 100%.

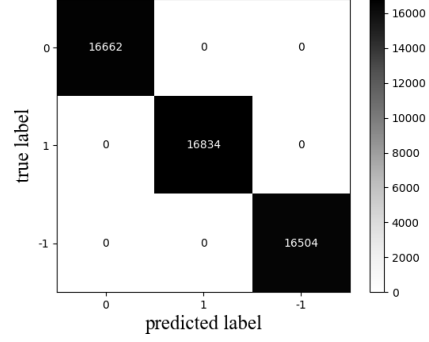
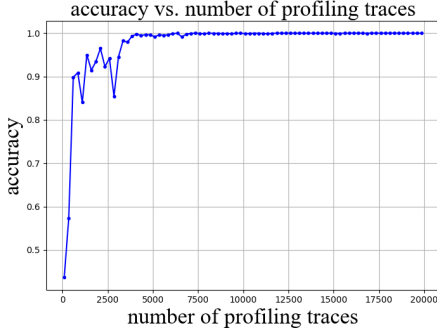
In the rest of this subsection, let B denote the input coefficients directly recovered by the first-layer leakage, and thus all the input upper input coefficients are contained in B , i.e., $\bigcup_{k=1}^n c_{n-k} \subseteq B$. However, since the Hamming weight leakage of 0 and 1 are very similar, it is challenging to distinguish the lower input coefficient value of 1 from 0 . To address this, we need to consider further leakages from other layers.

5.3.2 Exploiting leakage of the other layers

Given all the lower input coefficients with values of -1 and the values of upper input coefficients in the first layer, we propose the method to distinguish the remaining lower input coefficients whose values are within $\{0, 1\}$. Through the propagation of BFU in



(a) Profiling set size vs. attack accuracy of an upper input coefficient.


 (b) The confusion matrix of the upper input coefficient for distinguishing among the classes $\{0, 1, -1\}$


(c) Profiling set size vs. attack accuracy of a lower input coefficient.

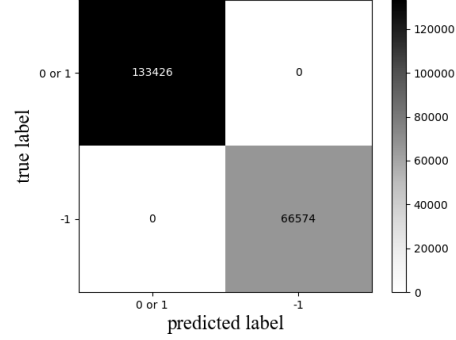

 (d) The confusion matrix of the lower input coefficient for distinguishing between the class $\{-1\}$ and $\{0, 1\}$.

Figure 4: Attack results of the first-layer NTT.

NTT, the differences in Hamming weights will be amplified, especially before and after the multiplication. Our attack exploits the leakages of $c_{i,j}$ and $c'_{i,j}$.

Through side-channel leakage, we can recover approximately the Hamming weight of each lower coefficient and its product with the corresponding twiddle factor in the 2nd to $(\log n)^{\text{th}}$ layers. We mean by ‘approximately’ that a tolerance ϵ is allowed between the actual Hamming weight and the predicted Hamming weight. In other words, through leakage, in 2nd to $(\log n)^{\text{th}}$ layers, we can recover that the Hamming weight of a lower coefficient (say, $c_{i,j}$) lies between $\text{HW}(c_{i,j}) - \epsilon$ to $\text{HW}(c_{i,j}) + \epsilon$, as well as the Hamming weight of $c'_{i,j}$ lies between $\text{HW}(c'_{i,j}) - \epsilon$ to $\text{HW}(c'_{i,j}) + \epsilon$. Specifically, we denote the approximate Hamming weight of a coefficient $c_{i,j}$ (resp., $c'_{i,j}$) with tolerance ϵ as $\text{HW}_\epsilon(c_{i,j})$ (resp., $\text{HW}_\epsilon(c'_{i,j})$).

The side-channel prediction accuracy of this approximation depends on the value of ϵ . Figure 5 illustrates the prediction accuracy relative to ϵ . We observe that when $\epsilon \geq 3$, the prediction accuracy reaches 100% (The model has epochs set to 20 and a batch size of 512). This implies that, under our attack setting, the tolerance ϵ is 3, as we are able to recover $\text{HW}_\epsilon(c_{i,j})$ with 100% accuracy for $\epsilon = 3$. In the rest of this subsection, we present the recovery of lower input coefficients $c_0, \dots, c_{\frac{n}{2}-1}$ using the approximate Hamming weights.

Upper coefficients $C_{i,j} \stackrel{\text{def}}{=} \{c_{f(i,j,k),j}, c'_{f(i,j,k),j} \mid k = 0, 1, 2, \dots, 2^{j-1} - 1, n - \frac{n}{2^j} \leq i < n\}$,

Algorithm 5 Attack with the approximate Hamming weights

```

1: Input:
2:   Side-channel leakage of NTT, MLP model, tolerance  $\epsilon$ .
3: Output:
4:   All input coefficients  $\{c_0, \dots, c_{255}\}$  or Unsuccess.
5:  $B \xleftarrow{MLP}$  Side-channel leakage
    $\triangleright$  B represents the input coefficients recovered by leakage of the first layer.
6: for  $j \leftarrow 2$  to  $\log n$  do  $\triangleright j$  represents the  $j^{\text{th}}$  layer in the NTT.
7:   for  $i \leftarrow n - \frac{n}{2^j}$  to  $n-1$  do
8:      $f(i, j, k) \stackrel{\text{def}}{=} i - \frac{k \cdot n}{2^{j-1}}, g(i, j) \stackrel{\text{def}}{=} i - \frac{(2^{j-1}-1) \cdot n}{2^{j-1}}$ 
9:      $C_{i,j} \stackrel{\text{def}}{=} \{c_{f(i,j,k),j}, c'_{f(i,j,k),j} \mid k = 0, 1, 2, \dots, 2^{j-1}-1\}$ 
10:     $A_{i,j} \stackrel{\text{def}}{=} \{c_{f(i,j,k)} \mid k = 0, 1, 2, \dots, 2^{j-1}-1\}$   $\triangleright$  Related input coefficients.
11:     $c_{g(i,j)} = A_{i,j}/A_{j-1}$   $\triangleright c_{g(i,j)}$  is the input coefficient to be recovered.
12:    if  $c_{g(i,j)} \in B$  then  $\triangleright$  Check if  $c_{g(i,j)}$  is recovered in the first layer.
13:       $c_{g(i,j)} \leftarrow -1$  and break
14:       $mark_1 \leftarrow \{False\}^2$   $\triangleright$  Indicate whether  $\tilde{c}_{g(i,j)}$  can be recovered.
15:       $mark_2 \leftarrow \{False\}^{2^{j-1}}$   $\triangleright$  Indicate whether  $c_{f(i,j,k),j}$  can be recovered.
16:      for  $\tilde{c}_{g(i,j)} \leftarrow 0$  to  $1$  do
17:         $\tilde{C}_{i,j} \stackrel{\text{def}}{=} \{\tilde{c}_{f(i,j,k),j}, \tilde{c}'_{f(i,j,k),j} \mid k = 0, 1, 2, \dots, 2^{j-1}-1\}$ 
    $\triangleright$  Coefficients derived from  $\tilde{c}_{g(i,j)}$  and  $A_{j-1}$  in  $j^{\text{th}}$  layer.
18:        for  $k \leftarrow 0$  to  $2^{j-1}-1$  do
19:           $HW_{\epsilon}(c_{f(i,j,k),j}), HW_{\epsilon}(c'_{f(i,j,k),j}) \xleftarrow{MLP}$  Side-channel leakage
20:           $HW(\tilde{c}_{f(i,j,k),j}), HW(\tilde{c}'_{f(i,j,k),j}) \xleftarrow{\tilde{c}_{g(i,j)}, A_{j-1}}$  Deduction
    $\triangleright$  Derive HW from guessed  $\tilde{c}_{g(i,j)}$  and previously recovered  $A_{j-1}$ .
21:          if  $|HW(\tilde{c}_{f(i,j,k),j}) - HW_{\epsilon}(c_{f(i,j,k),j})| \leq \epsilon$ 
22:            and  $|HW(\tilde{c}'_{f(i,j,k),j}) - HW_{\epsilon}(c'_{f(i,j,k),j})| \leq \epsilon$  then
    $\triangleright$  Check if HW is in the range of  $HW_{\epsilon}$ 
23:               $mark_2[k] \leftarrow True$ 
24:            if  $False \in mark_2$  then  $\triangleright$  If at least one HW is not in the range of  $HW_{\epsilon}$ .
25:              return Unsuccess
26:            else
27:               $mark_1[\tilde{c}_{g(i,j)}] \leftarrow True$ 
28:            if  $mark_1[0] == mark_1[1]$  then  $\triangleright$  There exist conflicting results.
29:              return Unsuccess
30:            else
31:               $c_{g(i,j)} \leftarrow (mark_1[0] == True) ? 0 : 1$ 
32:             $A_j \stackrel{\text{def}}{=} A_{j-1} \cup \bigcup_{k=1}^{\frac{n}{2^j}} A_{n-k,j}$   $\triangleright$  The set of recovered coefficients.
    $\triangleright A_{\log n} = \{c_1, c_2, \dots, c_{n-1}\}$ 
33:    if  $c_0 \in B$  then  $\triangleright$  Check if  $c_0$  is recovered in the first layer.
34:       $c_0 \leftarrow -1$ 
35:    else
36:       $c_0 \leftarrow (A_{\log n} \text{ contains } \tau \text{ nonzero input coefficients}) ? 0 : 1$ 
37:    return  $A_{\log n} \cup \{c_0\}$ 

```

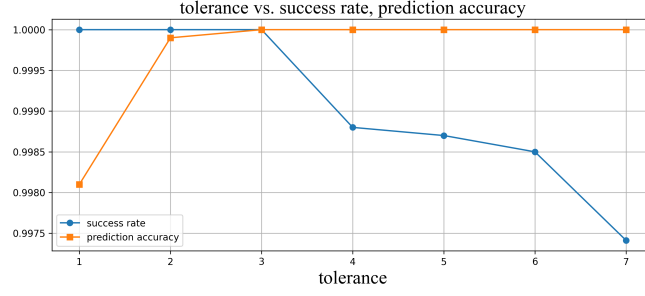


Figure 5: The relation between the calculated SR, prediction accuracy, and tolerance ϵ .

where $f(i, j, k) \stackrel{\text{def}}{=} i - \frac{k \cdot n}{2^{j-1}}$. These coefficients in the j^{th} layer are determined by the input coefficients $A_{i,j} \stackrel{\text{def}}{=} \{c_{i - \frac{k \cdot n}{2^{j-1}}} \mid k = 0, 1, \dots, 2^{j-1} - 1\}$. Let $A_j \stackrel{\text{def}}{=} A_{j-1} \cup \bigcup_{k=1}^{\frac{n}{2^j}} A_{n-k,j}$, so we have $A_{\log n} = \{c_1, c_2, \dots, c_{n-1}\}$, and $\{c_{i - \frac{(2^j-1) \cdot n}{2^j}}\} = A_{i,j}/A_{j-1}$.

Assuming A_{j-1} is known, it is possible to resolve the input coefficients $c_{g(i,j)} = A_{i,j}/A_{j-1}$, with $g(i, j) \stackrel{\text{def}}{=} i - \frac{(2^{j-1}-1) \cdot n}{2^{j-1}}$ by a probability. It can be achieved by exhaustively guessing the value of $c_{g(i,j)}$, calculating the corresponding Hamming weights of the coefficients in $C_{i,j}$, and verifying whether the computed Hamming weights fall within the expected range of their approximated Hamming weights. Therefore, we can execute the attack by leveraging the leakage from each successive layer. Algorithm 5 describes the process of this attack.

In the j^{th} iteration in the algorithm (lines 10 to 30), A_{j-1} is already recovered by the previous layers. Then, for the lower input coefficient $c_{g(i,j)}$, if $c_{g(i,j)} = -1$, then $c_{g(i,j)} \in B$ is recovered in the first layer. Otherwise, we take a guess value $\tilde{c}_{g(i,j)} \in \{0, 1\}$, and calculate the coefficients in $C_{i,j}$ through the NTT route, denoted as $\tilde{C}_{i,j} \stackrel{\text{def}}{=} \{\tilde{c}_{f(i,j,k),j}, \tilde{c}'_{f(i,j,k),j} \mid k = 0, 1, 2, \dots, 2^{j-1} - 1\}$. If all the Hamming weights of the coefficients in $\tilde{C}_{i,j}$ fall within the range of approximated Hamming weights of the coefficients in $C_{i,j}$, the guessed value $\tilde{c}_{g(i,j)}$ may be correct. However, note that the two guess values 0 and 1 may both exhibit to be correct ones because of the too-large tolerance ϵ of the approximated Hamming weights. We regard this case as an unsuccessful attack.

At last, the only undetermined input coefficient is c_0 , which can be obtained based on the value of τ . Concretely, since the number τ of nonzero input coefficients in $c_1, \dots, c_{n-1}$ is known, we can directly ascertain whether or not c_0 is zero. Additionally, as we already know, if c_0 is -1 or not, its exact value can be determined.

The success rate of recovering all input coefficients using the above attack method depends on the value of the tolerance ϵ . To evaluate this, we perform the attack with different ϵ values and calculate the success rate by iterating the attack 40 million times for each ϵ value. Figure 5 shows the relationship between the success rate, the prediction accuracy of the SCA model, and the tolerance ϵ . As observed, both the attack success rate (of Algorithm 5) and the prediction accuracy of approximate Hamming weight reach 100% when $\epsilon = 3$. To better illustrate Algorithm 5, we provide an example based on NTT with 16 input coefficients in Appendix B.

5.4 Recovering $\mathbf{z}$

In Algorithm 3, line 11, $\mathbf{z}$ is calculated by $\mathbf{z} = \mathbf{y} + \mathbf{c}\mathbf{s}_1$ in the polynomial ring modulo q . By exploiting the leakage of the $\mathbf{z}$ using DL profiled attacks, we can determine whether a given coefficient of $\mathbf{z}$ takes one of the following critical values: $-\gamma_1, -\gamma_1 - 1, \gamma_1 - 1, \gamma_1$, or any other value. Notably, a rejection occurs if any coefficient of $\mathbf{z}$ hits these values during the Dilithium signing process. We use leakage from three functions to perform the

classification.

5.4.1 Exploiting leakage of poly_chknorm

```

1 int poly_chknorm(const poly *a, int32_t B) {
2     unsigned int i;
3     int32_t t;
4     DBENCH_START();
5     if(B > (Q-1)/8)
6         return 1;
7     for(i = 0; i < N; ++i) {
8         /* Absolute value */
9         t = a->coeffs[i] >> 31;
10        t = a->coeffs[i] - (t & 2*a->coeffs[i]);
11        if(t >= B) {
12            DBENCH_STOP(*tsample);
13            return 1;
14        }
15    }
16    DBENCH_STOP(*tsample);
17    return 0;
18 }

```

Figure 6: The C implementation of “poly_chknorm”.

In the Dilithium reference implementation, the “poly_chknorm” function in Figure 6 checks if $\mathbf{z}$ satisfies $\|\mathbf{z}\|_\infty \leq \gamma_1 - \beta$. The reference implementation employs an early abort strategy, specifically, it immediately rejects and restarts the signing process when encountering invalid $\mathbf{z}$ ’s coefficients, thus achieving 100% accuracy in determining coefficient rejection. The power consumption leakage from both the absolute value computation and the if-statement comparison in Figure 6 can distinguish $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, γ_1 , or any other rejected values. Thus, we perform the classification task using the aforementioned power consumption leakage and the MLP model from Table 5.

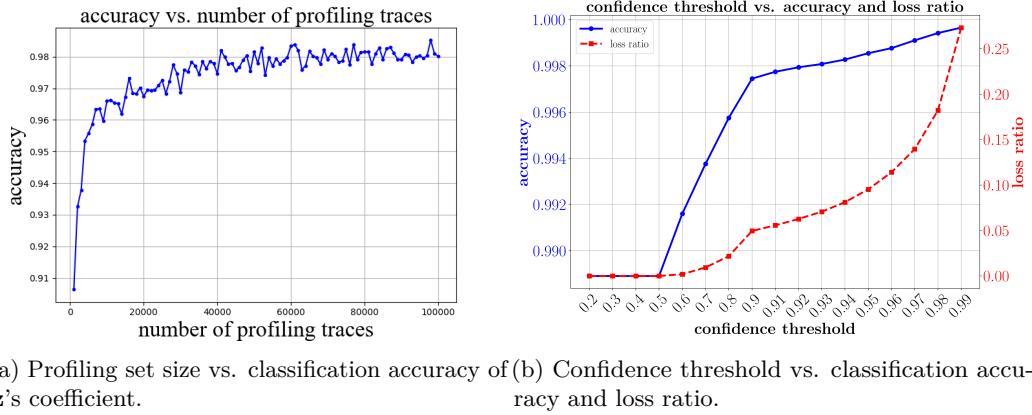


Figure 7: Attack result of function “poly_chknorm”.

Figure 7a shows the model’s (epochs=60, batch size=512) prediction accuracy (for distinguishing $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, γ_1 , or other rejected values) with the number of profiling traces. The prediction accuracy fluctuates around 98%. Since our attack requires 100% prediction accuracy, we implement a simple filtering scheme. The Softmax function is applied in the last layer of the MLP model, which converts the raw scores of the network’s

output into probabilities between 0 and 1. These probabilities indicate prediction confidence. We use a confidence threshold to filter predictions. Only predictions with confidence above the threshold are accepted. We use 200,000 traces to determine the optimal confidence threshold. This method improves prediction accuracy at the cost of sacrificing some power traces. Figure 7b quantifies how the confidence threshold affects accuracy and loss ratio. Specifically, define N_{all} as the count of $\mathbf{z}$ coefficients in the attack set that equal $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, or γ_1 , and N_{pre} as the corresponding predicted count. The loss ratio is calculated as $\frac{N_{\text{all}} - N_{\text{pre}}}{N_{\text{all}}}$.

5.4.2 Exploiting leakage of poly_add

<pre> 1 void poly_add(poly *c, const poly *a, const poly *b){ 2 unsigned int i; 3 DBENCH_START(); 4 for(i = 0; i < N; ++i) 5 c->coeffs[i] = a->coeffs[i] + b->coeffs[i]; 6 DBENCH_STOP(&tadd); 7 } </pre>	<pre> 1 polyvecl_pointwise_poly_montgomery(&z, &cp, &s1); 2 polyvecl_invntt_tomont(&z); 3 polyvecl_add(&z, &z, &y); 4 polyvecl_reduce(&z); 5 6 if(polyvecl_chknorm(&z, GAMMA1 - BETA)) 7 goto rej; </pre>
---	---

Figure 8: The reference implementation of “poly_add”.

As shown in Figure 8, since $\mathbf{z} = \mathbf{y} + \text{INTT}(\hat{c} \cdot \hat{s}_1)$, the leakage from the addition operation classifies $\mathbf{z}$ ’s coefficients. Similarly, the MLP in Table 5 distinguishes (the model is trained for 60 epochs using a batch size of 512) whether $\mathbf{z}$ ’s coefficients are $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, γ_1 , or other rejected values. Following Section 5.4.1, we collect the power trace of this function for rejected $\mathbf{z}$ ’s coefficients. Additionally, the classification of “poly_add” can also adopt the confidence threshold filtering scheme mentioned in Section 5.4.1. The results are shown in Figure 9.

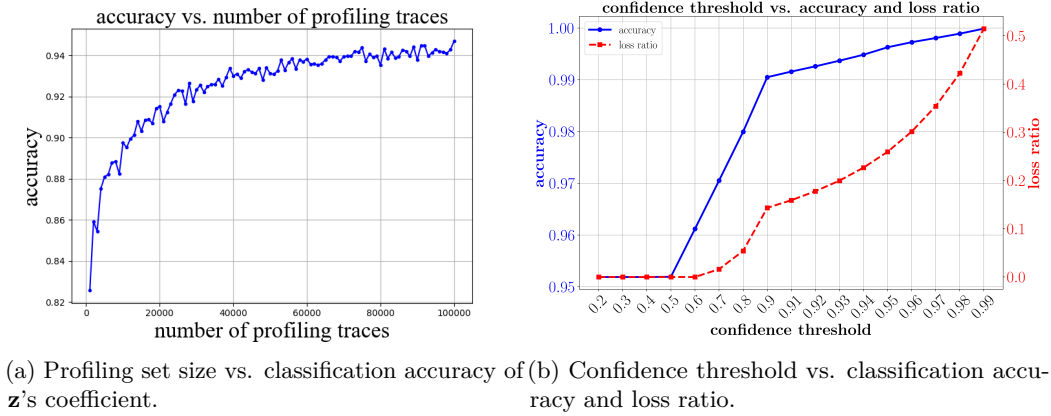


Figure 9: Attack result of function “poly_add”.

5.4.3 Exploiting leakage of reduce32

<pre> 1 int32_t reduce32(int32_t a) { 2 int32_t t; 3 t = (a + (1 << 22)) >> 23; 4 t = a - t*Q; 5 return t; 6 } </pre>	<pre> 1 polyvecl_pointwise_poly_montgomery(&z, &cp, &s1); 2 polyvecl_invntt_tomont(&z); 3 polyvecl_add(&z, &z, &y); 4 polyvecl_reduce(&z); 5 if(polyvecl_chknorm(&z, GAMMA1 - BETA)) 6 goto rej; </pre>
---	---

Figure 10: The reference implementation of “reduce32”.

As shown in Figure 10, since $\mathbf{z} = \mathbf{y} + \text{INTT}(\hat{c} \cdot \hat{s}_1)$, the Dilithium reference implementation performs a lazy reduction (modular reduction) after the addition, implemented by the core function “reduce32”. We classify $-\gamma_1$, $-\gamma_1 - 1$, $\gamma_1 - 1$, γ_1 , or other values using leakage from “reduce32” and the MLP model in Table 5 (The model is trained for 60 epochs using a batch size of 512). The dataset construction follows the same approach as in Section 5.4.1. Similarly, the classification of “reduce32” can also adopt the confidence threshold filtering scheme mentioned in Section 5.4.1. The results are shown in Figure 11.

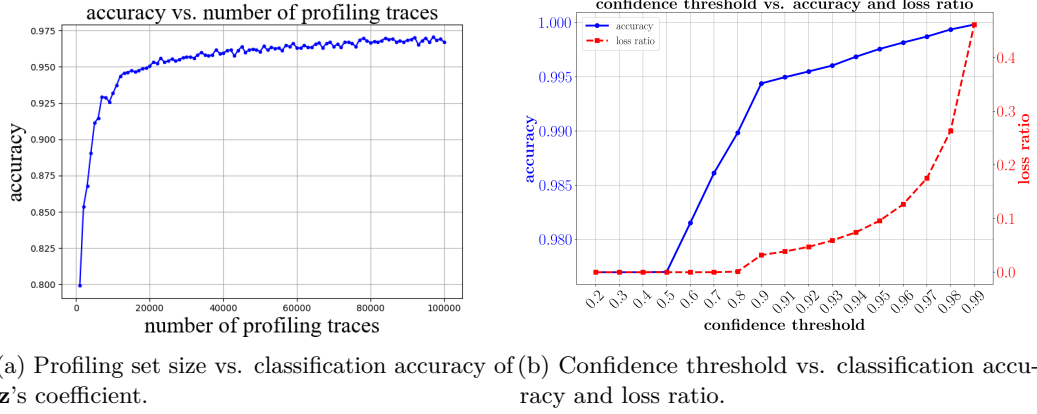


Figure 11: Attack result of function “reduce32”.

5.4.4 Attack algorithm using combined leakage

To ensure prediction accuracy, we combine leakage from “poly_chknorm”, “poly_add”, and “reduce32”, with aforementioned confidence threshold filtering scheme. We set multiple thresholds for filtering and thus design Algorithm 6. Here we define the prediction results of “poly_chknorm”, “poly_add”, and “reduce32” as Z_{chk} , Z_{add} , and Z_{red} , respectively. Additionally, we define $\text{threshold}_{C\&R}$ as the confidence threshold for “poly_chknorm” and “reduce32”, threshold_A as the confidence threshold for “poly_add”, threshold_1 as the confidence threshold for predictions being γ_1 or $-\gamma_1 - 1$, and threshold_2 as the confidence threshold for predictions being $\gamma_1 - 1$ or $-\gamma_1$.

This filtering scheme ensures that the prediction accuracy approaches 100%. We collect 5 million traces of rejected $\mathbf{z}$ ’s coefficient, and 100% accuracy means that each of those traces is either discarded (due to the confidence threshold) or classified correctly. With $\text{threshold}_{C\&R_1} = 0.95$, $\text{threshold}_{C\&R_2} = 0.994$, $\text{threshold}_{A_1} = 0.98$ and $\text{threshold}_{A_2} = 0.999$ the prediction results are shown in the confusion matrix in Figure 12, achieving a loss ratio of 0.457.

5.5 Result analysis

Section 5.3 presents a method to recover c ’s coefficient with 100% SR per trace. For the recovery of $\mathbf{z}$ ’s coefficient, the method presented in Sections 5.4 achieves 100% true positive rate at the cost of sacrificing some traces (with a certain loss ratio), making the required practical signature count become larger than N_{sig} . However, Section 4 considers an ideal condition (i.e., no data loss and 100% true positive rate) and shows that the $N_{\text{sig}} = 3 \times 10^6$ (resp., 2.3×10^7 and 1.3×10^7) signature generation trials are sufficient to recover $\mathbf{s}_1$ for security level 2 (resp., 3 and 5) with a 100% SR. Considering the loss ratio in $\mathbf{z}$ ’s coefficient recovery, the practical number of required signatures becomes $\frac{N_{\text{sig}}}{1 - \text{loss_ratio}} = \frac{N_{\text{sig}}}{1 - 0.457}$, which is 5.525×10^6 (resp., 4.259×10^7 and 2.394×10^7) for security level 2 (resp., 3 and 5).

Algorithm 6 Classify coefficients of $\mathbf{z}$

```

1: Input:
2:   MLP models, leakage of “poly_chknorm”, “poly_add”, and “reduce32”, thresholdC&R1, thresholdC&R2,
   thresholdA1, thresholdA2.
3: Output:
4:   Classification of a coefficient of  $\mathbf{z}$  into  $-\gamma_1, -\gamma_1 - 1, \gamma_1 - 1, \gamma_1$  or others.
5: if The coefficient of  $\mathbf{z}$  is rejected then ▷ Early abortion.
6:    $Z_{chk} \xleftarrow{MLP_c}$  Side-channel leakage ▷ Classify “poly_chknorm” leakage.
7:    $Z_{add} \xleftarrow{MLP_a}$  Side-channel leakage ▷ Classify “poly_add” leakage.
8:    $Z_{red} \xleftarrow{MLP_r}$  Side-channel leakage ▷ Classify “reduce32” leakage.
9:    $Mark \leftarrow \{False\}^3$  ▷ A record on whether the confidence of the predictions meets the thresholds.
10:  ▷ When the 3 prediction results are inconsistent, discard the trace.
11:  if  $Z_{chk} == Z_{add} == Z_{red}$  then
12:     $Z = Z_{chk}$ 
13:  else
14:    Discard trace
15:  ▷ Determine if confidence > threshold for each case.
16:  if  $z \in \{-\gamma_1, \gamma_1 - 1\}$  then
17:    if Confidence( $Z_{chk}$ ) > thresholdC&R1 then
18:       $Mark[0] \leftarrow True$ 
19:    if Confidence( $Z_{red}$ ) > thresholdC&R1 then
20:       $Mark[1] \leftarrow True$ 
21:    if Confidence( $Z_{add}$ ) > thresholdA1 then
22:       $Mark[2] \leftarrow True$ 
23:  else
24:    if Confidence( $Z_{chk}$ ) > thresholdC&R2 then
25:       $Mark[0] \leftarrow True$ 
26:    if Confidence( $Z_{red}$ ) > thresholdC&R2 then
27:       $Mark[1] \leftarrow True$ 
28:    if Confidence( $Z_{add}$ ) > thresholdA2 then
29:       $Mark[2] \leftarrow True$ 
30:    ▷ Since “poly_add” has a large loss ratio, we use an OR relation here to reduce the loss.
31:  if  $Mark[0] \wedge Mark[1] == True$  or  $Mark[2] == True$  then
32:     $\mathbf{z}$ 's coefficient  $\leftarrow Z$ 
33:  else
34:    Discard trace
    
```

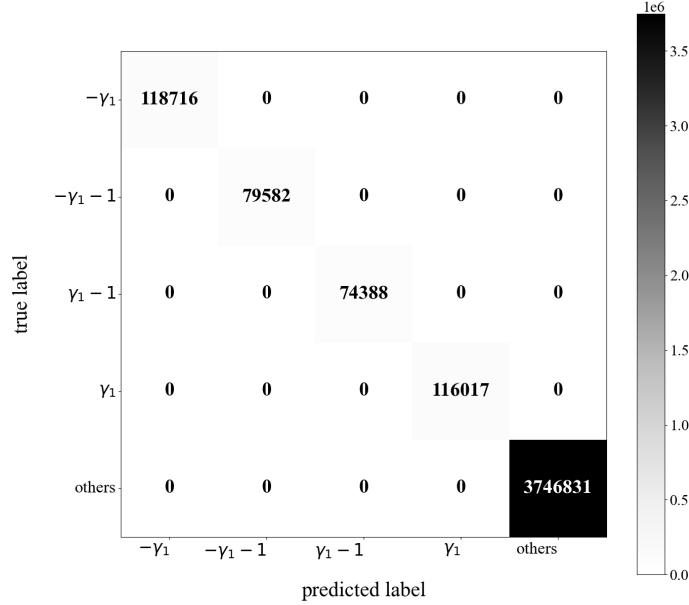


Figure 12: Attack result using combined leakage.

5.6 Countermeasures

There are different potential countermeasures to eliminate or mitigate this threat. Masking countermeasures proposed in recent literature, including but not limited to [BBE⁺18, MGTF19, ABC⁺23, CGL⁺24], are expected to provide provably secure protection against this rejected signature's challenge attack. However, as mentioned before, the disadvantage of masking countermeasures is the performance penalty.

Apart from the masking, one can also implement classical hiding countermeasures, e.g., increasing the noise level during the bound check of $\mathbf{z}$ and handling of c . It can be done by adding time noise (for instance, variable clock, jitters, random delay) or balancing power consumption while processing different target data ($\mathbf{z}$ and c in this case) values. A shuffling countermeasure can be another way to mitigate the attack [RPBC20, LHL⁺23], i.e., using a shuffled order to process $\mathbf{z}$ and c . Compared to masking, hiding countermeasures can mitigate security risks but may not completely eliminate them. For instance, an adversary could potentially recover the shuffled index, thereby bypassing the shuffling countermeasure. In conclusion, a combination of those mitigation countermeasures can render the attack complexity much higher, while it can only mitigate, but might not fix, the potential security issue completely. It should be noted that, evaluating these countermeasures is beyond the scope of this paper, and we leave it as a promising direction for future work. Furthermore, we believe our attack underscores the critical importance of thoroughly protecting the challenge and response in the implementation of Dilithium.

6 Conclusion and future work

Existing SCA research on ML-DSA/Dilithium mainly focuses on direct leakages of sensitive data, such as the nonce $\mathbf{y}$, polynomial multiplications $\mathbf{cs}_1$ and $\mathbf{cs}_2$, and the decomposed commitment $\mathbf{w}_0$. The rejection sampling mechanism prevents secret key leakage by discarding responses $\mathbf{z}$ that fail the bound check.

In this work, we introduced a rejected signature's challenge attack to recover the private key $\mathbf{s}_1$, exploiting side-channel leakages from rejected $\mathbf{z}$ values and their corresponding challenges c . This is formulated as an ILP problem with bounded $\mathbf{cs}_1$ constraints derived from the rejected $\mathbf{z}$ and c .

Our experiments across all security levels of Dilithium-2/3/5 validate the effectiveness of this approach, demonstrating the feasibility of private key recovery with sufficient rejected $\mathbf{z}_i$ and challenge pairs. Real-world experiments on Dilithium reference implementation on a 32-bit ARM Cortex-M4 further confirm the practicality of this attack.

As a possible future direction, we plan to enhance the attack's tolerance to generic SCA errors in the recovered $\mathbf{z}_i$ and c 's coefficients, particularly in low-SNR scenarios, where only a single attack trace is available. Additionally, we should investigate countermeasures to mitigate the proposed attacks and strengthen the security of real-world implementations.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (Grant No. 2020YFA0309705), the National Natural Science Foundation of China (Grant Nos. 62125204, 92270201 and 62372273), the Program of Taishan Young Scholars of the Shandong Province, Innovation Program for Quantum Science and Technology (Grant No. 2021ZD0302901 / 2021ZD0302902). Yu Yu's work has also been supported by the New Corner-stone Science Foundation through the XPLOER PRIZE.

References

- [ABC⁺23] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Tobias Schneider, Markus Schönauer, François-Xavier Standaert, and Christine van Vredendaal. Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023, Issue 4:58–79, 2023.
- [AOVCG25] Paco Azevedo-Oliveira, Andersson Calle Viera, Benoît Cogliati, and Louis Goubin. Finding a polytope: A practical fault attack against dilithium. Cryptology ePrint Archive, Paper 2025/195, 2025.
- [BAE⁺24] Olivier Bronchain, Melissa Azouaoui, Mohamed ElGhamrawy, Joost Renes, and Tobias Schneider. Exploiting small-norm polynomial multiplication with physical attacks: Application to crystals-dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 024 No. 2:359–383, 2024.
- [BBD⁺23] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. Springer-Verlag, 2023.
- [BBE⁺18] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the glp lattice-based signature scheme at any order. In *EUROCRYPT (2)*, pages 354–384. Springer, 2018.
- [BDK⁺20] Shi Bai, Láo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS–dilithium: Algorithm specification and supporting documentation (version 3.1). Round-3 :ubmission to the NIST Post-Quantum Cryptography Standardization Project, 2020. <https://cryptojedi.org/papers/#dilithiumnistr3>.
- [BG14] Shi Bai and Steven D. Galbraith. An improved compression technique for signatures based on learning with errors. In Josh Benaloh, editor, *Topics in Cryptology - CT-RSA 2014 - The Cryptographer’s Track at the RSA Conference 2014, San Francisco, CA, USA, February 25-28, 2014. Proceedings*, volume 8366 of *Lecture Notes in Computer Science*, pages 28–47. Springer, 2014.
- [Bot24] Botan. Botan, a Crypto and TLS for Modern C++ library, Version: 3.5.0. Available at https://github.com/randombit/botan/blob/master/src/lib/pubkey/dilithium/dilithium_common/dilithium_algos.cpp, 2024.
- [BVC⁺23] Alexandre Berzati, Andersson Calle Viera, Maya Chartouny, Steven Madec, Damien Vergnaud, and David Vigilant. Exploiting intermediate value leakage in dilithium: A template-based approach. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023, Issue 4:188–210, 2023.
- [CGL⁺24] Jean-Sébastien Coron, François Gérard, Tancrede Lepoint, Matthias Trannoy, and Rina Zeitoun. Improved high-order masked generation of masking vector and rejection sampling in dilithium. *IACR Cryptol. ePrint Arch.*, page 1149, 2024.

- [CKA⁺21] Zhaohui Chen, Emre Karabulut, Aydin Aysu, Yuan Ma, and Jiwu Jing. An efficient non-profiled side-channel attack on the crystals-dilithium post-quantum signature. In *39th IEEE International Conference on Computer Design, ICCD 2021, Storrs, CT, USA, October 24-27, 2021*, pages 583–590. IEEE, 2021.
- [CRR02] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template attacks. In *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*, pages 13–28. Springer, 2002.
- [DFMS19] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the fiat-shamir transformation in the quantum random-oracle model. 11693:356–383, 2019.
- [DFPS23] Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of fiat-shamir with aborts. Springer-Verlag, 2023.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *Transactions on Cryptographic Hardware and Embedded Systems*, 2018, Issue 1:238–268, 2018.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.
- [FSK03] Daniel L. Fults, Michael K. Staton, and Leonard J. Kazmier. *Business Statistics : Based on Schaum's Outline of Theory and Problems of Business Statistics*. McGraw-Hill, 2003.
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [GJ79] M. R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [HNP25] Julius Hermelink, Kai-Chun Ning, and Richard Petri. Finding and protecting the weakest link: On side-channel attacks on masked ML-DSA. *IACR Cryptol. ePrint Arch.*, page 276, 2025.
- [JMW24] Kelsey A. Jackson, Carl A. Miller, and Daochen Wang. Evaluating the security of crystals-dilithium in the quantum random oracle model. Springer-Verlag, 2024.
- [Jr.83] Hendrik W. Lenstra Jr. Integer programming with a fixed number of variables. *Math. Oper. Res.*, 8(4):538–548, 1983.
- [KAA21] Emre Karabulut, Erdem Alkim, and Aydin Aysu. Single-trace side-channel attacks on ω -small polynomial sampling: With applications to ntru, NTRU prime, and CRYSTALS-DILITHIUM. In *IEEE International Symposium on Hardware Oriented Security and Trust, HOST 2021, Tysons Corner, VA, USA, December 12-15, 2021*, pages 35–45. IEEE, 2021.
- [Kel60] Henry J Kelley. Gradient theory of optimal flight paths. *Ars Journal*, 30(10):947–954, 1960.

- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.
- [KLH⁺20] Il-Ju Kim, Taeho Lee, Jaeseung Han, Bo-Yeon Sim, and Dong-Guk Han. Novel single-trace ML profiling attacks on NIST 3 round candidate dilithium. *IACR Cryptol. ePrint Arch.*, page 1383, 2020.
- [KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In *EUROCRYPT (3)*, pages 552–586. Springer, 2018.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [LHL⁺23] Jonghyeok Lee, Jaeseung Han, Sangyub Lee, Jihoon Kwon, Keon-Hee Choi, Jae-Won Huh, Jihoon Cho, and Dong-Guk Han. Systematization of shuffling countermeasures: With an application to crystals-dilithium. *IEEE Access*, 11:142862–142873, 2023.
- [Lib24] Liboqs. Liboqs, the open quantum safe quantum-safe cryptographic algorithms library, Version: 0.10.1. Available at https://github.com/open-quantum-safe/liboqs/blob/main/src/sig/ml_dsa/pqcrystals-dilithium-standard_ml-dsa-44-ipd_ref/poly.c, 2024.
- [LLZ⁺24] Yong Liu, Yuejun Liu, Yongbin Zhou, Yiwen Gao, Zehua Qiao, and Huaxin Wang. A novel power analysis attack against crystals-dilithium implementation. In *IEEE European Test Symposium, ETS 2024, The Hague, Netherlands, May 20-24, 2024*, pages 1–6. IEEE, 2024.
- [LSMK63] John D. C. Little, Katta G. Murty, Dura W. Sweeney, and Caroline Karel. An algorithm for the traveling salesman problem. *Operations Research*, 11(6):972–989, 1963.
- [Lyu09] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer, 2009.
- [Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755. Springer, 2012.
- [LZ19] Qipeng Liu and Mark Zhandry. Revisiting post-quantum fiat-shamir. 11693:326–355, 2019.

- [LZS⁺21] Yuejun Liu, Yongbin Zhou, Shuo Sun, Tianyu Wang, Rui Zhang, and Jingdian Ming. On the security of lattice-based fiat-shamir signatures in the presence of randomness leakage. *IEEE Trans. Inf. Forensics Secur.*, 16:1868–1879, 2021.
- [MGTF19] Vincent Migliore, Benoît Gérard, Mehdi Tibouchi, and Pierre-Alain Fouque. Masking dilithium - efficient implementation and side-channel evaluation. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *Applied Cryptography and Network Security - 17th International Conference, ACNS 2019, Bogota, Colombia, June 5-7, 2019, Proceedings*, volume 11464 of *Lecture Notes in Computer Science*, pages 344–362. Springer, 2019.
- [NIS24] NIST. FIPS 204, Module-Lattice-Based Digital Signature Standard (FIPS 204). Available at <https://doi.org/10.6028/NIST.FIPS.204>, 2024.
- [PVG⁺11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [QLZ⁺23a] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Jingdian Ming, Chengbin Jin, and Huizhong Li. Practical public template attack attacks on crystals-dilithium with randomness leakages. *IEEE Trans. Inf. Forensics Secur.*, 18:1–14, 2023.
- [QLZ⁺23b] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Mingyao Shao, and Shuo Sun. When NTT meets SIS: efficient side-channel attacks on dilithium and kyber. *IACR Cryptol. ePrint Arch.*, page 1866, 2023.
- [QLZ⁺24] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Yuhan Zhao, and Shuyi Chen. Single trace is all it takes: Efficient side-channel attack on dilithium. *IACR Cryptol. ePrint Arch.*, page 512, 2024.
- [RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan-Pieter D’Anvers, and Anubhab Baksı. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. *ACM Trans. Embed. Comput. Syst.*, 23(2):35:1–35:54, 2024.
- [RJH⁺18] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Side-channel assisted existential forgery attack on dilithium - A NIST PQC candidate. *IACR Cryptol. ePrint Arch.*, page 821, 2018.
- [RPBC20] Prasanna Ravi, Romain Poussier, Shivam Bhasin, and Anupam Chattopadhyay. On configurable SCA countermeasures against single trace attacks for the NTT - A performance evaluation study over kyber and dilithium on the ARM cortex-m4. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering - 10th International Conference, SPACE 2020, Kolkata, India, December 17-21, 2020, Proceedings*, volume 12586 of *Lecture Notes in Computer Science*, pages 123–146. Springer, 2020.
- [RR23] Victor Reis and Thomas Rothvoss. The subspace flatness conjecture and faster integer programming. In *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*, pages 974–988. IEEE, 2023.

- [Sch89] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252. Springer, 1989.
- [SLKG23] Hauke Malte Steffen, Georg Land, Lucie Johanna Kogelheide, and Tim Güneysu. Breaking and protecting the crystal: Side-channel analysis of dilithium in hardware. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023, College Park, MD, USA, August 16-18, 2023, Proceedings*, volume 14154 of *Lecture Notes in Computer Science*, pages 688–711. Springer, 2023.
- [TMS24] Tolun Tosun, Amir Moradi, and Erkey Savas. Exploiting the central reduction in lattice-based cryptography. *IACR Cryptol. ePrint Arch.*, page 66, 2024.
- [TS24] Tolun Tosun and Erkey Savas. Zero-value filtering for accelerating non-profiled side-channel attack on incomplete ntt-based implementations of lattice-based cryptography. *IEEE Trans. Inf. Forensics Secur.*, 19:3353–3365, 2024.
- [UMTS22] Vincent Quentin Ulitzsch, Soundes Marzougui, Mehdi Tibouchi, and Jean-Pierre Seifert. Profiling side-channel attacks on dilithium - A small bit-fiddling leak breaks it all. In Benjamin Smith and Huapeng Wu, editors, *Selected Areas in Cryptography - 29th International Conference, SAC 2022, Windsor, ON, Canada, August 24-26, 2022, Revised Selected Papers*, volume 13742 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2022.
- [vzGS78] Joachim von zur Gathen and Malte Sieveking. A bound on solutions of linear integer equalities and inequalities. *Proceedings of the American Mathematical Society*, 72(1):155–158, 1978.
- [WGL⁺24] Huaxin Wang, Yiwen Gao, Yuejun Liu, Qian Zhang, and Yongbin Zhou. In-depth correlation power analysis attacks on a hardware implementation of crystals-dilithium. *Cybersecur.*, 7(1):21, 2024.
- [WNGD23] Ruize Wang, Kalle Ngo, Joel Gärtner, and Elena Dubrova. Single-trace side-channel attacks on crystals-dilithium: Myth or reality? *IACR Cryptol. ePrint Arch.*, page 1931, 2023.

Appendix

A Concrete Hamming weights of rejected coefficient $\mathbf{z}_i$

For each security level of Dilithium-2/3/5, we analyzed the histogram of the Hamming weights of those $4\beta + 1$ possible values of rejected coefficient $\mathbf{z}_i$. For Dilithium-2, HW-15/31/21/25 are unique ones when the rejected $\mathbf{z}_i$ is in the interval of $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$, however, HW-21/25 were never observed during our analyses of the distribution of $\mathbf{x}_i$ shown in 1a. Thus, only HW-15/31 corresponding to $-\gamma_1$ and $-\gamma_1 - 1$ in this interval are considered for our rejected signature’s challenge attack in this work. Similarly, HW-17/1/7/11 are unique ones when the rejected $\mathbf{z}_i$ is in the interval of $[\gamma_1 - \beta, \gamma_1 + \beta]$, however, HW-7/11 were never observed during our analyses of the distribution of $\mathbf{x}_i$ shown in 1a. Thus, only HW-17/1 corresponding to $\gamma_1 - 1$ and γ_1 in this interval are considered for our rejected signature’s challenge attack in this work.

For Dilithium-3/5, only HW-13/31 corresponding to $-\gamma_1$ and $-\gamma_1 - 1$ in the interval of $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$, and HW-19/1 corresponding to $\gamma_1 - 1$ and γ_1 in the interval

of $[\gamma_1 - \beta, \gamma_1 + \beta]$ are considered for our rejected signature's challenge attack. Because they are the only ones with unique HW values in those two intervals, respectively.

We summarized the histogram of HW values of those $4\beta + 1$ possible values of rejected coefficient $\mathbf{z}_i$ in Tables 6 to 11.

Table 6: HW corresponding to 2β possible rejected $\mathbf{z}_i$ coefficient in the interval of $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$ for Dilithium-2

HW	15	16	17	18	19	20	21	25	26	27	28	29	30	31
Frequency	1	7	19	26	18	6	1	1	6	17	26	19	7	1

Table 7: HW corresponding to $2\beta + 1$ possible rejected $\mathbf{z}_i$ coefficient in the interval of $[\gamma_1 - \beta, \gamma_1 + \beta]$ for Dilithium-2

HW	1	2	3	4	5	6	7	11	12	13	14	15	16	17
Frequency	1	7	19	26	19	6	1	1	6	18	26	19	7	1

Table 8: HW corresponding to 2β possible rejected $\mathbf{z}_i$ coefficient in the interval of $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$ for Dilithium-3

HW	13	14	15	16	17	18	19	20	24	25	26	27	28	29	30	31
Frequency	1	8	28	52	56	36	13	2	2	13	36	55	52	28	8	1

Table 9: HW corresponding to $2\beta + 1$ possible rejected $\mathbf{z}_i$ coefficient in the interval of $[\gamma_1 - \beta, \gamma_1 + \beta]$ for Dilithium-3

HW	1	2	3	4	5	6	7	8	12	13	14	15	16	17	18	19
Frequency	1	8	28	53	56	36	13	2	2	13	36	56	52	28	8	1

Table 10: HW corresponding to 2β possible rejected $\mathbf{z}_i$ coefficient in the interval of $[-\gamma_1 + 1 - \beta, -\gamma_1 + 1 + \beta]$ for Dilithium-5

HW	13	14	15	16	17	18	19	25	26	27	28	29	30	31
Frequency	1	7	21	35	34	18	4	3	18	34	35	21	7	1

Table 11: HW corresponding to $2\beta + 1$ possible rejected $\mathbf{z}_i$ coefficient in the interval of $[\gamma_1 - \beta, \gamma_1 + \beta]$ for Dilithium-5

HW	1	2	3	4	5	6	7	13	14	15	16	17	18	19
Frequency	1	7	21	35	35	18	4	4	18	34	35	21	7	1

B An attack example for Algorithm 5

To better illustrate Algorithm 5, we provide an example based on NTT with 16 input coefficients, shown in Figure 3. Our example traces the process from the 1st to the final layer of the NTT.

- **1st layer:** In conjunction with the description in Section 5.3.1, we can leverage the side-channel leakage from the first layer to recover the upper input coefficients of the first layer BFU (i.e., $\{c_8, c_9, \dots, c_{15}\}$), as well as the lower input coefficients whose values are -1 . It recovers input coefficients in the set B . The subsequent attacks are conducted based on the method described in Section 5.3.2.

- 2nd layer:** Taking $i = 15$ as a case example, set $C_{15,2} = \{c_{15,2}, c'_{15,2}, c_{7,2}, c'_{7,2}\}$ is a function of input coefficients $A_{15,2} = \{c_{15}, c_7\}$, where $c_{15} \in A_1$ has already been determined during the attack process in the previous (i.e., the first) layer. If $c_7 = -1$, then $c_7 \in B$ is also known from the previous layer. If $c_7 \neq -1$, then we have $A_{15,2}/A_1 = c_7$ and attempt to recover this input coefficient through the side-channel leakage of the second layer. For a hypothesis $\tilde{c}_7 \in \{0, 1\}$ of c_7 , we can compute the set $\tilde{C}_{15,2} = \{\tilde{c}_{15,2}, \tilde{c}'_{15,2}, \tilde{c}_{7,2}, \tilde{c}'_{7,2}\}$ by $\tilde{c}_7$ and c_{15} ($c_{15} \in A_1$) along the NTT path. Thus, we can compute the predicted Hamming weights of $\tilde{c}_{15,2}$, $\tilde{c}'_{15,2}$, $\tilde{c}_{7,2}$, and $\tilde{c}'_{7,2}$. Additionally, we can obtain the approximate Hamming weights of those four coefficients by exploiting the side-channel leakage. If and only if all predicted guessed Hamming weights match the corresponding approximate Hamming weights, the guess $\tilde{c}_7$ is deemed valid. Furthermore, the attack is successful if only one guessed value of c_7 is valid. If both hypotheses $\tilde{c}_7 = 0$ and $\tilde{c}_7 = 1$ are valid, this situation is considered an attack failure. Through the above attack scheme for the second layer attack, we can recover c_7 , c_6 , c_5 , and c_4 , so we have $A_2 = A_1 \cup A_{15,2} \cup A_{14,2} \cup A_{13,2} \cup A_{12,2}$, or equivalently, i.e., $A_2 = A_1 \cup \{c_7, c_6, c_5, c_4\}$.
- 3rd layer:** Taking $i = 15$ as a case example, set $C_{15,3} = \{c_{f(15,k,3),3}, c'_{f(15,k,3),3} \mid k = 0, 1, 2, 3\}$ is a function of input coefficients $A_{15,3} = \{c_{15}, c_{11}, c_7, c_3\}$, where $\{c_{15}, c_{11}, c_7\} \in A_2$ have been determined during the attack process in the second layer. If $c_3 = -1$, then $c_3 \in B$ is known in the first layer. If $c_3 \neq -1$, we have $A_{15,3}/A_2 = c_3$, and we attempt to recover this input coefficient through the side-channel leakage of the 3rd layer. For a hypothesis $\tilde{c}_3 \in \{0, 1\}$ of c_3 , we can compute the set $\tilde{C}_{15,3} = \{\tilde{c}_{f(15,k,3),3}, \tilde{c}'_{f(15,k,3),3} \mid k = 0, 1, 2, 3\}$ by $\tilde{c}_3$ and $\{c_{15}, c_{11}, c_7\} \in A_2$ along the NTT path. The subsequent attack process is similar to the description of attacking c_7 in the second layer. In the third layer attack, we can recover c_3 and c_2 , so we have $A_3 = A_2 \cup \bigcup_{k=1}^2 A_{n-k,3}$, or equivalently i.e. $A_3 \cup \{c_3, c_2\}$.
- 4th layer:** Taking $i = 15$ as a case example, set $C_{15,4} = \{c_{f(15,k,4),4}, c'_{f(15,k,4),4} \mid k = 0, 1, 2, \dots, 7\}$ is a function of input coefficients $A_{15,4} = \{c_{15}, c_{13}, c_{11}, c_9, c_7, c_5, c_3, c_1\}$, where $\{c_{15}, c_{13}, c_{11}, c_9, c_7, c_5, c_3\} \in A_3$ have been determined during the attack process in the third layer. If $c_1 = -1$, we know $c_1 \in B$ in the first layer. If $c_1 \neq -1$, we have $A_{15,4}/A_3 = c_1$ and attempt to recover this input coefficient through the side-channel leakage of the 4th layer. For a hypothesis $\tilde{c}_1 \in \{0, 1\}$ of c_1 , we can compute the set $\tilde{C}_{15,4} = \{\tilde{c}_{f(15,k,4),4}, \tilde{c}'_{f(15,k,4),4} \mid k = 0, 1, 2, \dots, 7\}$ by $\tilde{c}_1$ and $\{c_{15}, c_{13}, c_{11}, c_9, c_7, c_5, c_3\} \in A_3$ along the NTT path. The subsequent attack process is similar to the description of attacking c_7 in the second layer. In the fourth layer attack, we can recover c_1 , so we have $A_4 = A_3 \cup \bigcup_{k=1}^4 A_{n-k,4}$, i.e. $A_4 \cup \{c_1\}$. At this point, we can recover all input coefficients involved in the multiplications within the NTT, with the only undetermined input coefficient being c_0 . We can obtain its value through the τ value. If we have recovered the τ nonzero input coefficients in the polynomial c from $c_1, c_2, \dots, c_{15}$, then $c_0 = 0$. Additionally, since we already know if c_0 is -1 or not, its exact value can be determined.